Pimpri Chinchwad Education Trust's

Pimpri Chinchwad College of Engineering and Research, Ravet, Pune

Department of Computer Engineering

A.Y. 2025-26 - Term II

Unit 1

Introduction to Database Management System

SE Computer Engineering (2024 PAT)

Content

- Introduction to Database Management Systems,
- Purpose of Database Systems,
- Database-System Applications,
- View of Data,
- Database Languages,
- Database System Structure,
- Enterprise Constraints Data Models, Database Design and
- ER Model: Entity, Attributes, Relationships, Constraints, Keys, Design Process, Entity Relationship Model, ER Diagram, Design Issues, Extended E-R Features, Converting E-R & EER diagram into tables.
- Case Study: Study of Architecture of any DBMS like Oracle or MySQL. Design a database schema for any problem given in previous Question Papers.

Database Management System (DBMS)

- DBMS contains information about a particular enterprise
  - Collection of interrelated data
  - Set of programs to access the data
  - An environment that is both *convenient* and *efficient* to use

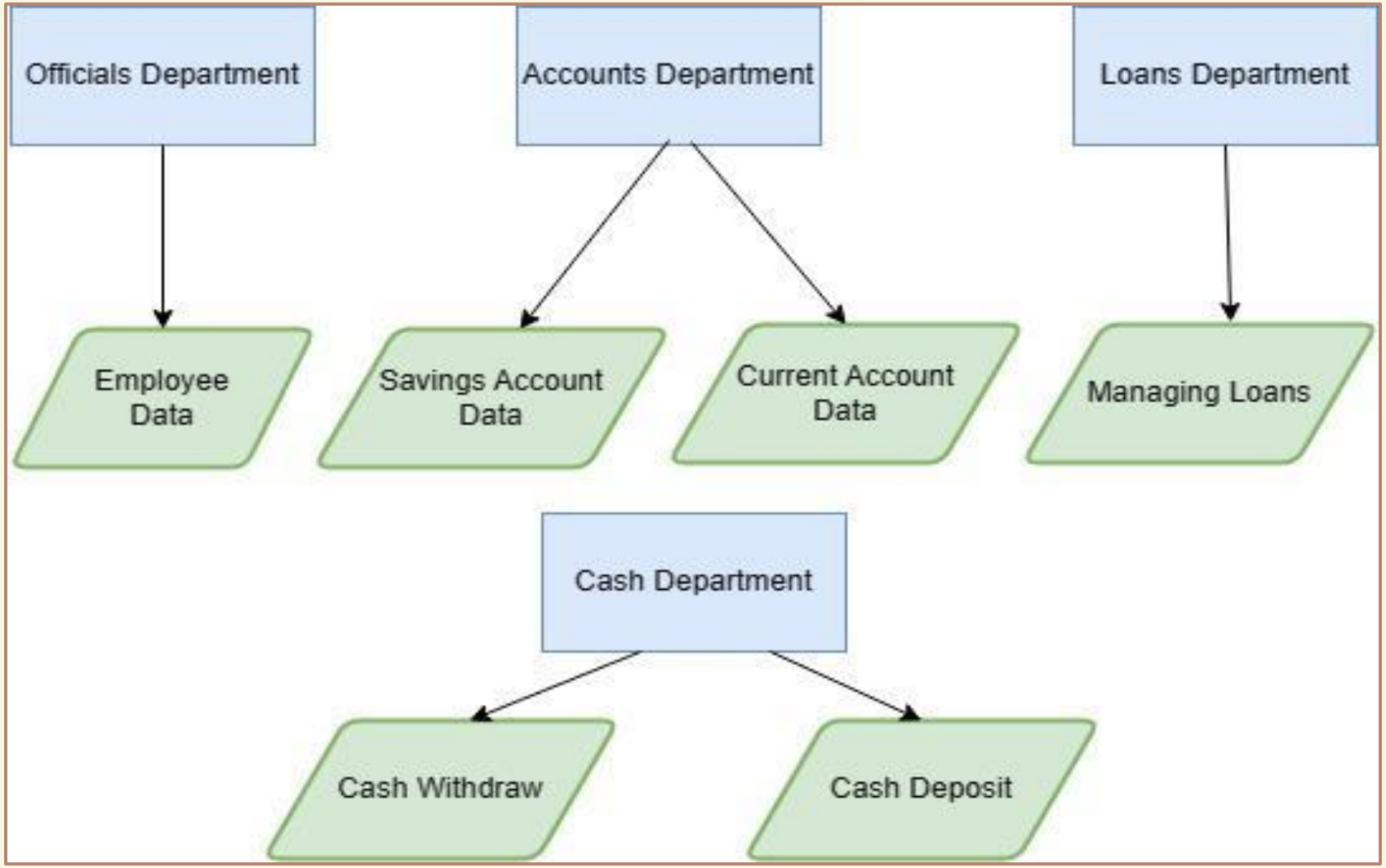
- Databases can be very large.
- Databases touch all aspects of our lives

University Database Example

- Application program examples
  - Add new students, instructors, and courses
  - Register students for courses, and generate class rosters
  - Assign grades to students, compute grade point averages (GPA) and generate transcripts
- In the early days, database applications were built directly on top of file systems

- **Databases are powerful tools *for storing and managing structured data.***
- ***They provide a structured and organized approach to handle vast amounts of information.***

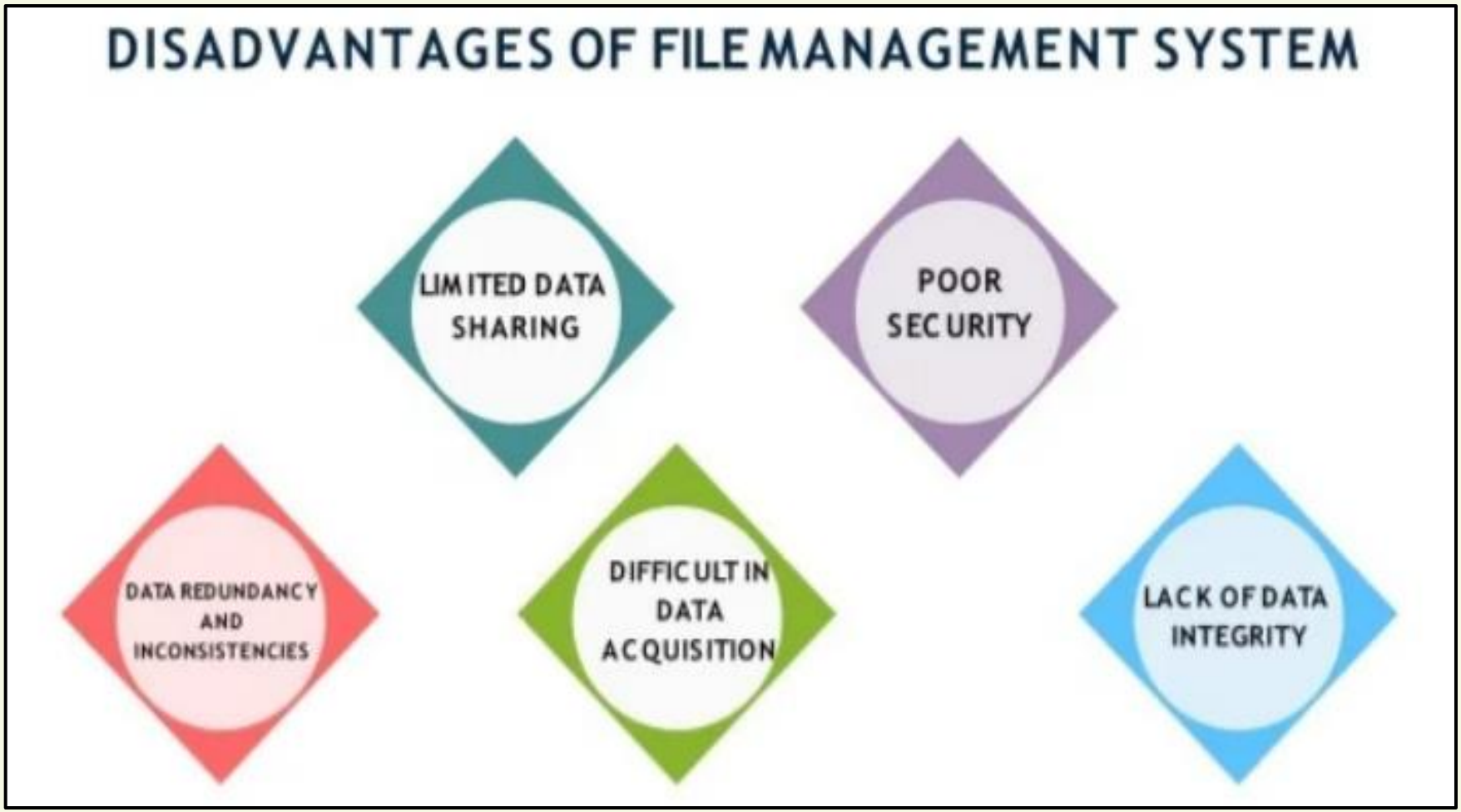
File System


Officials Department
Accounts Department
Loans Department
Employee Data
Savings Account Data
Current Account Data
Managing Loans
Cash Department
Cash Withdraw
Cash Deposit

File System (Example)

students.txt → Student
detailsmarks.txt → Exam
marksfees.txt → Fee payment records

- **Data Redundancy**: Student name and ID repeated in every file
- **Inconsistency**: Name updated in students.txt but not in fees.txt
- **Poor Security**: Anyone with file access can modify data
- **Slow Queries**: Finding "students who paid fees and passed exams" requires manual file processing
- **No Recovery**: If the system crashes, data may be lost permanently


DISADVANTAGES OF FILE MANAGEMENT SYSTEM
LIMITED DATA SHARING
POOR SECURITY
DATA REDUNDANCY AND INCONSISTENCIES
DIFFICULT IN DATA ACQUISITION
LACK OF DATA INTEGRITY

Drawbacks of using file systems to store data

- **Data redundancy:** Data redundancy refers to the duplication of data. Data redundancy often leads to higher storage costs and poor access time.
- **Data inconsistency:** Data redundancy leads to data inconsistency.

  E.g. A student is enrolled for two courses and we have student address stored twice, now lets say student requests to change his address, if the address is changed at one place and not on all the records then this can lead to data inconsistency.
- **Data Isolation:** Because data are scattered in various files, and files may be in different formats, writing new application programs to retrieve the appropriate data is difficult.

Drawbacks of using file systems to store data (Cont.)

- **Dependency on application programs:** Changing files would lead to change in application programs.
- **Atomicity issues:** Atomicity of a transaction refers to "All or nothing", which means either all the operations in a transaction executes or none.

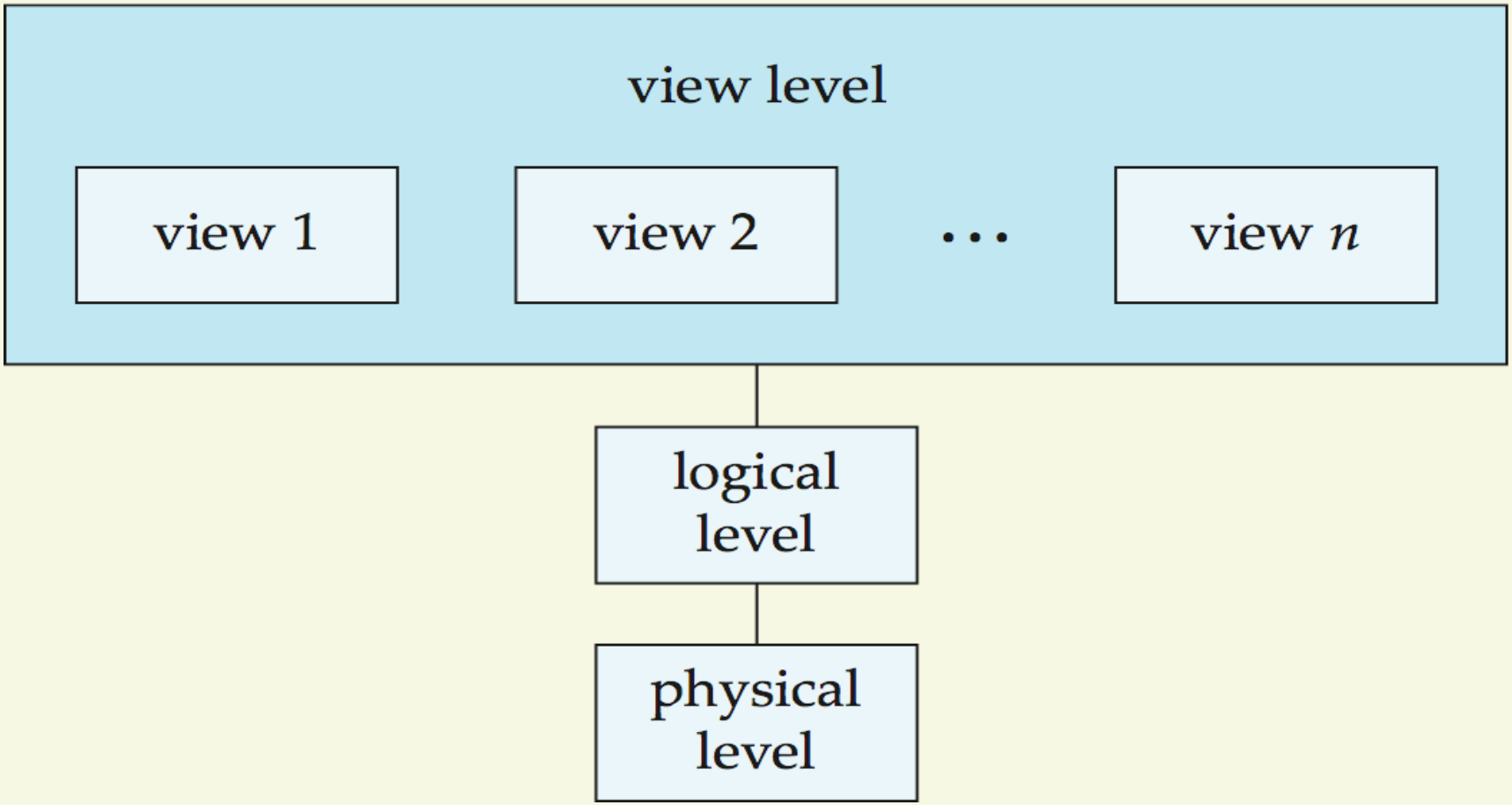
  In such case the rollback of operation should occur to maintain the atomicity of transaction. It is **difficult to achieve atomicity in file processing systems**.
- **Data Security:** Data should be secured from unauthorized access.

  E.g. A student in a college should not be able to see the payroll details of the teachers, such kind of security constraints are difficult to apply in file processing systems.

Database systems offer solutions to all the above problems

Database-System Applications

Levels of Abstraction



Levels of Abstraction

- **Physical level:** describes how a record (e.g., instructor) is stored.
- **Logical level:** describes data stored in database, and the relationships among the data.

```
type instructor = record
        ID : string;
        name : string;
        dept_name : string;
        salary : integer;
    end;
```

- **View level:** application programs hide details of data types. Views can also hide information (such as an employee's salary) for security purposes.

Instances and Schemas

- A *schema* is a description of a particular collection of data, using the a given data model.

- **Instance** – the actual content of the database at a particular point in time
  - Analogous to the value of a variable

- Applications insulated from how data is structured and stored.
- *Logical data independence*: Protection from changes in *logical* structure of data.
- *Physical data independence*: Protection from changes in *physical* structure of data.

☛ ***One of the most important benefits of using a DBMS!***

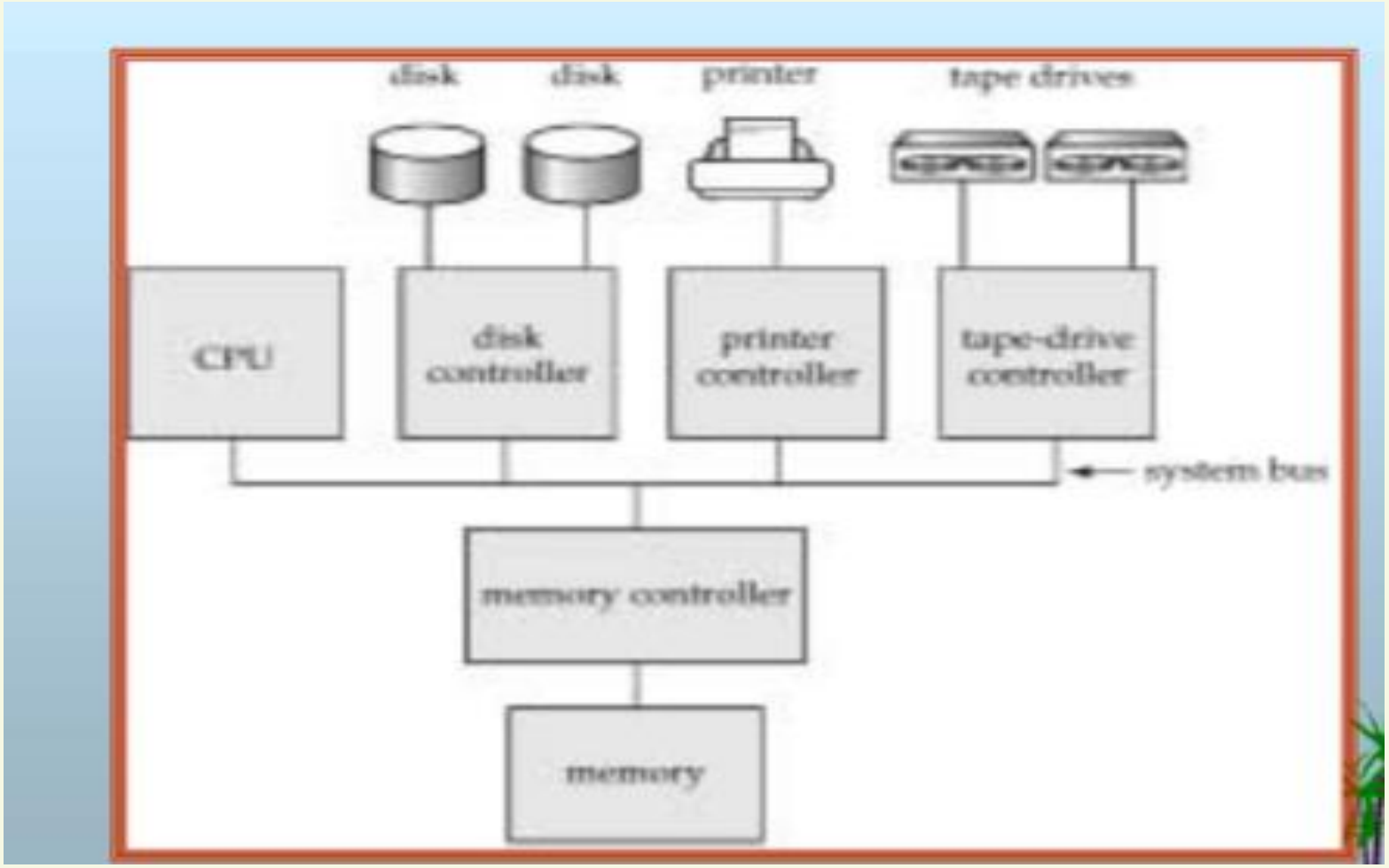
Database System Structure

Database Architecture

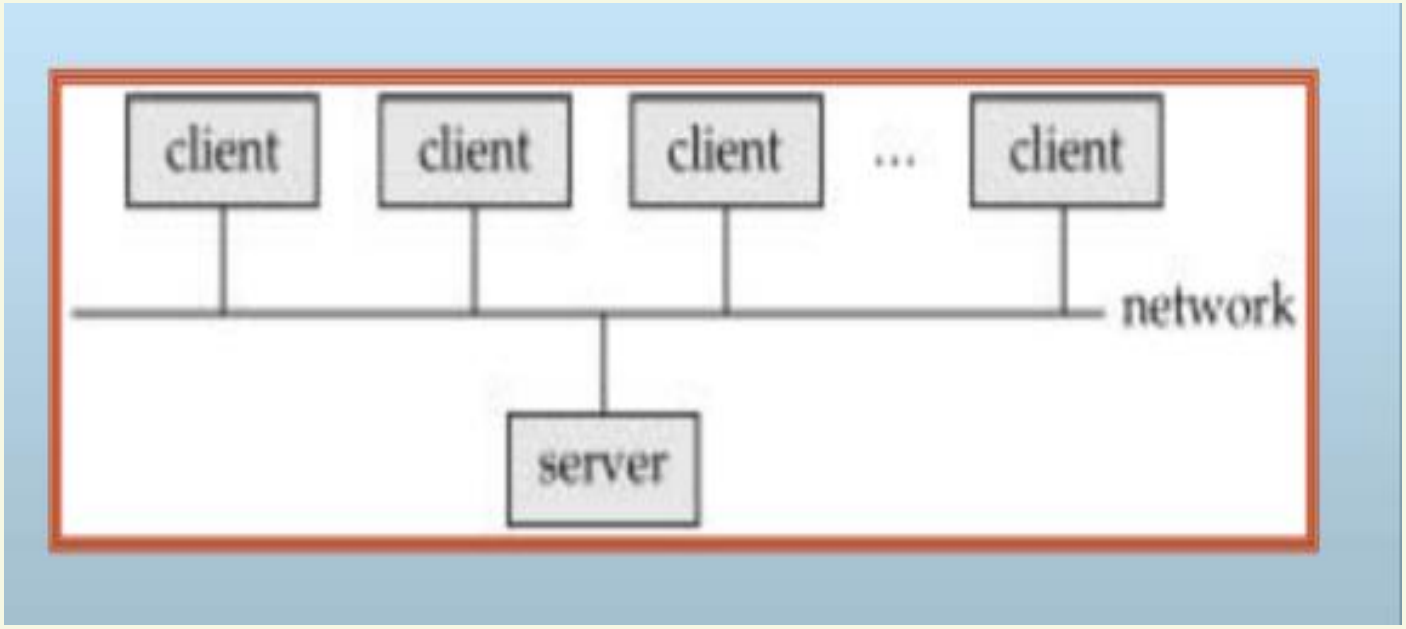
The architecture of a database systems is greatly influenced by the underlying computer system on which the database is running:

- Centralized
- Client-server
- Parallel (multi-processor)
- Distributed

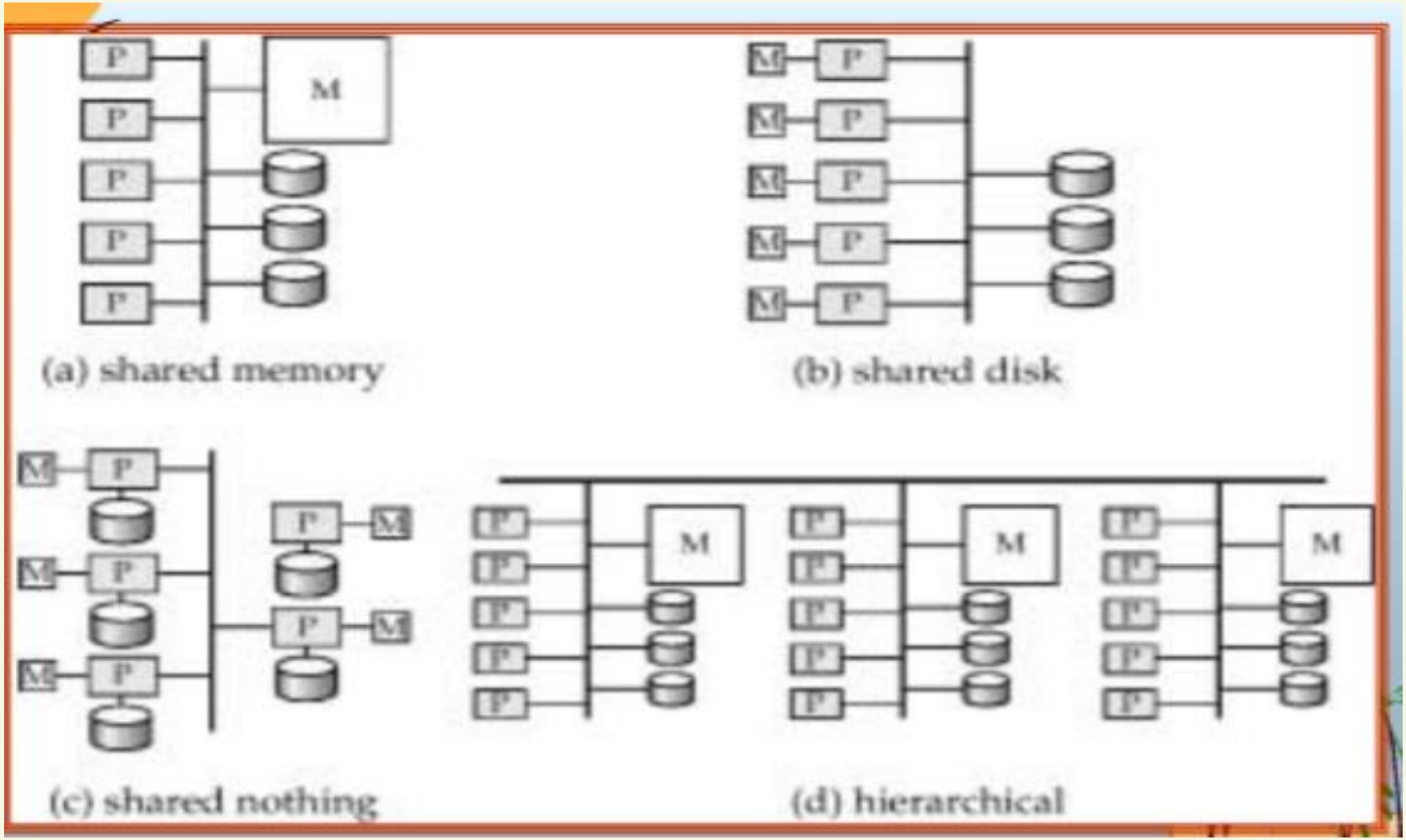
Centralized Database Architecture



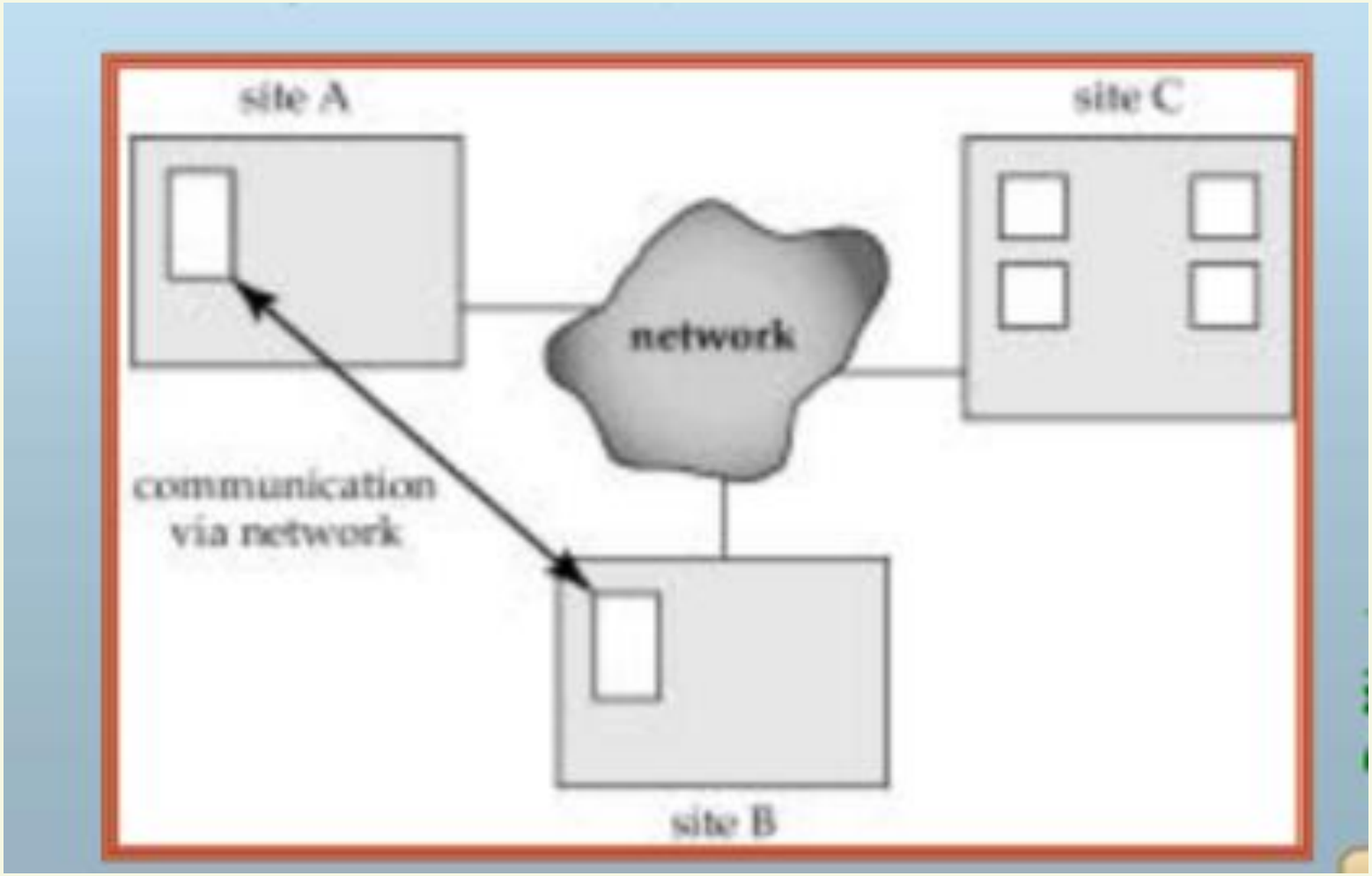
Client-Server Database Architecture



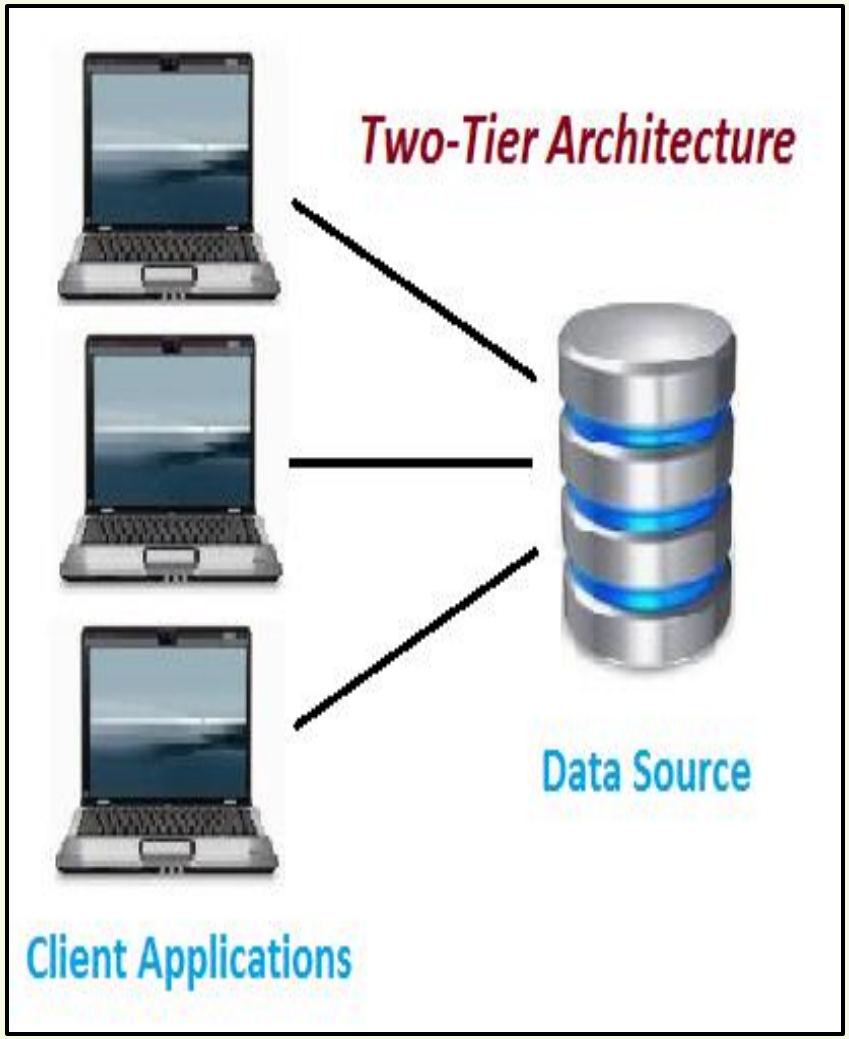
Parallel Database Architecture

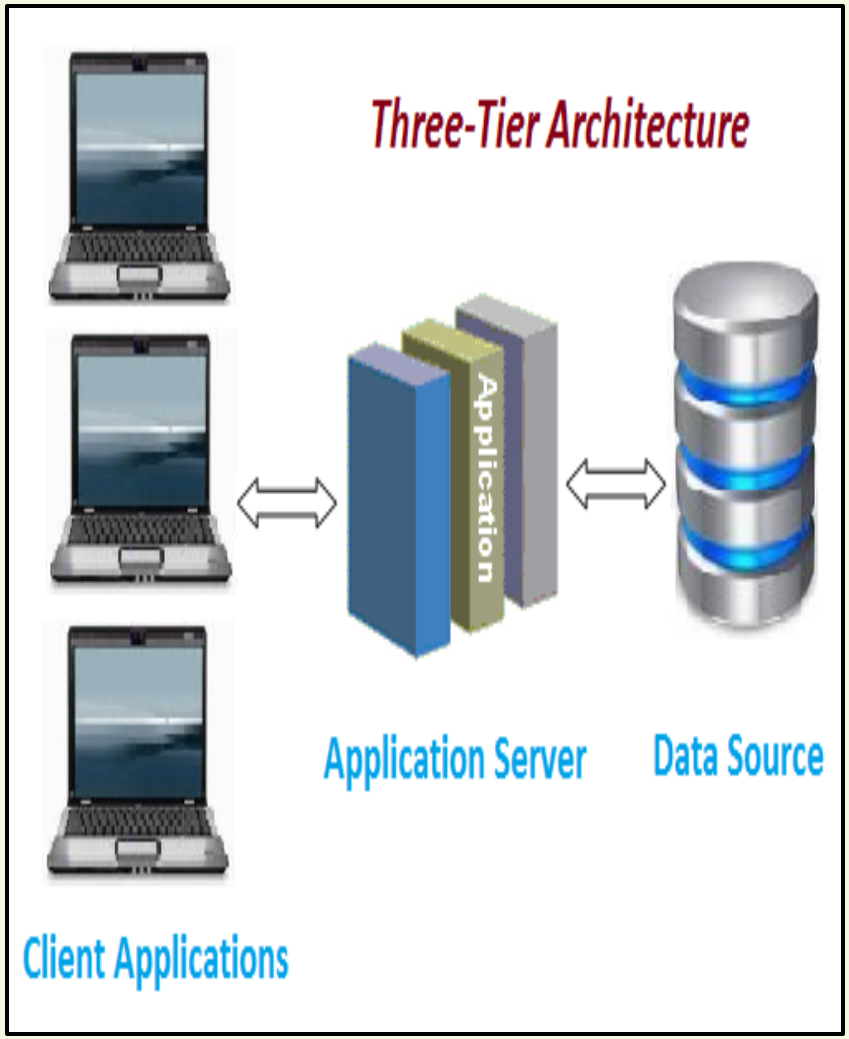


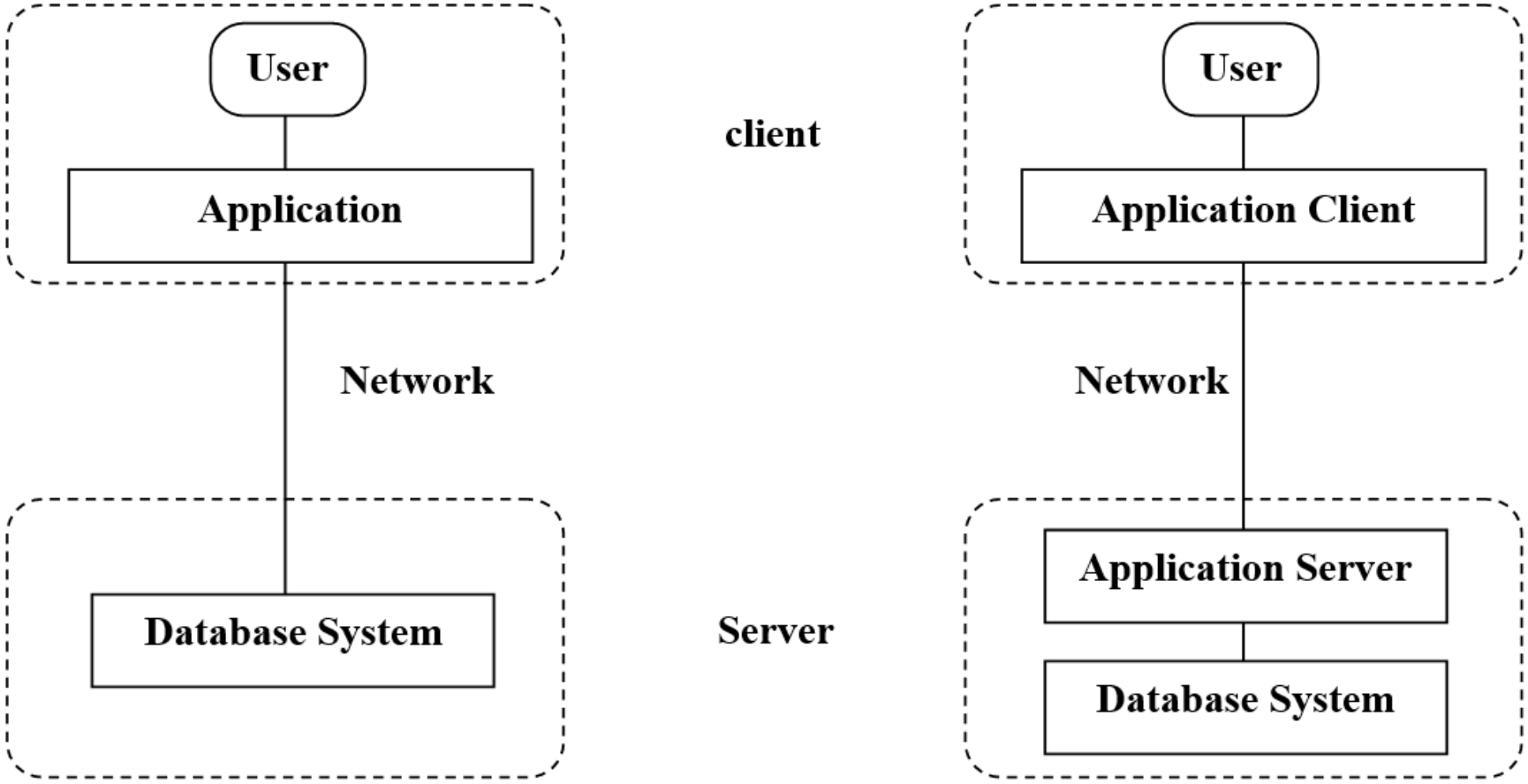
Distributed Database Architecture



Tier Architecture



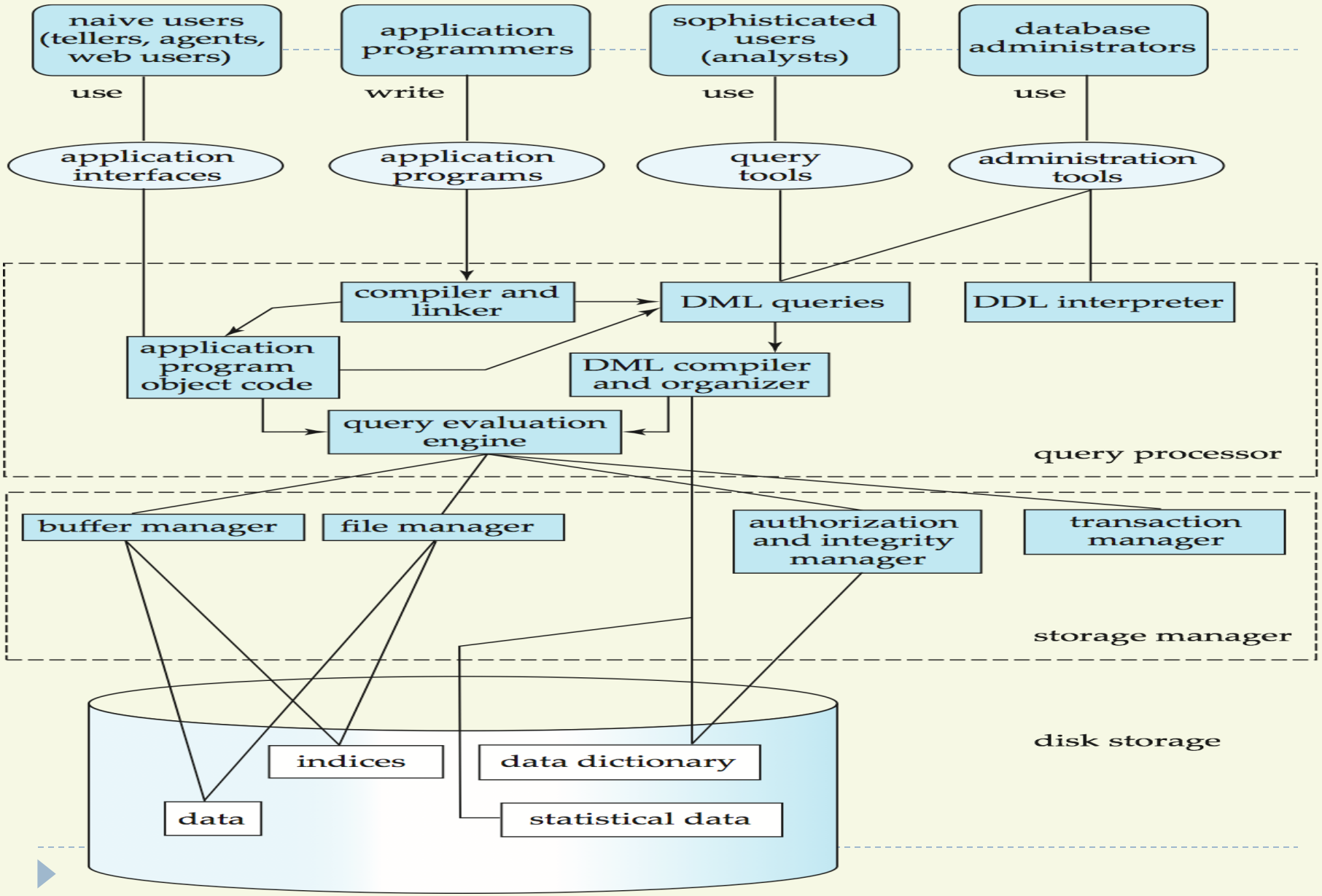




(a) Two-tier Architecture

(a) Three-tier Architecture

Overall Database System Structure



Overall Database System Structure

- **Database Architecture** refers to the internal components of the DBMS, including the Query Processor, Storage Manager, and Disk Storage.
- It also defines the interaction of these components.

Components of a Database System

- Query Processor, Storage Manager, and Disk Storage.

Query Processor

- It interprets the requests (queries) received from end user via an application program into instructions.
- It also executes the user request which is received from the **DML compiler**.

Query Processor contains the following components -

- **DML Compiler:** It processes the DML statements into low level instruction (machine language), so that they can be executed.
- **DDL Interpreter:** It processes the DDL statements into a set of table containing meta data (data about data).
- **Embedded DML Pre-compiler:** It processes DML statements embedded in an application program into procedural calls.
- **Query Optimizer:** The Query Optimizer executes instructions generated by the DML Compiler and improves query execution efficiency by choosing the best query plan, considering factors such as indexing, join order, and available system resources.

For instance, if a query involves joining two large tables, the optimizer will select the **best join order to minimize query execution time**.

Storage Manager (Database Control System)

- It is an interface between the data stored in the database and the queries received.
- It maintains the consistency and integrity of the database by applying the constraints and executing the DCL statements.
- It is responsible for updating, storing, deleting, and retrieving data in the database. It contains the following components:
- **Authorization Manager:** It ensures role-based access control, i.e,. checks whether the particular person is privileged to perform the requested operation or not.
- **Integrity Manager:** It checks the integrity constraints when the database is modified.
- **Transaction Manager:** It controls concurrent access by performing the operations in a scheduled way that it receives the transaction. File Manager: It manages the file space and the data structure used to represent information in the database.
- **Buffer Manager:** It is responsible for cache memory and the transfer of data between the secondary storage and main memory.

Disk Storage

- **Data Files:** It stores the actual data in the database.
- **Data Dictionary:** It contains the information about the **structure of database objects** such as tables, constraints, and relationships. It is the repository of information that governs the metadata.
- **Indices:** Provides **faster data retrieval** by allowing the DBMS to find records quickly, improving query performance.

Database Languages

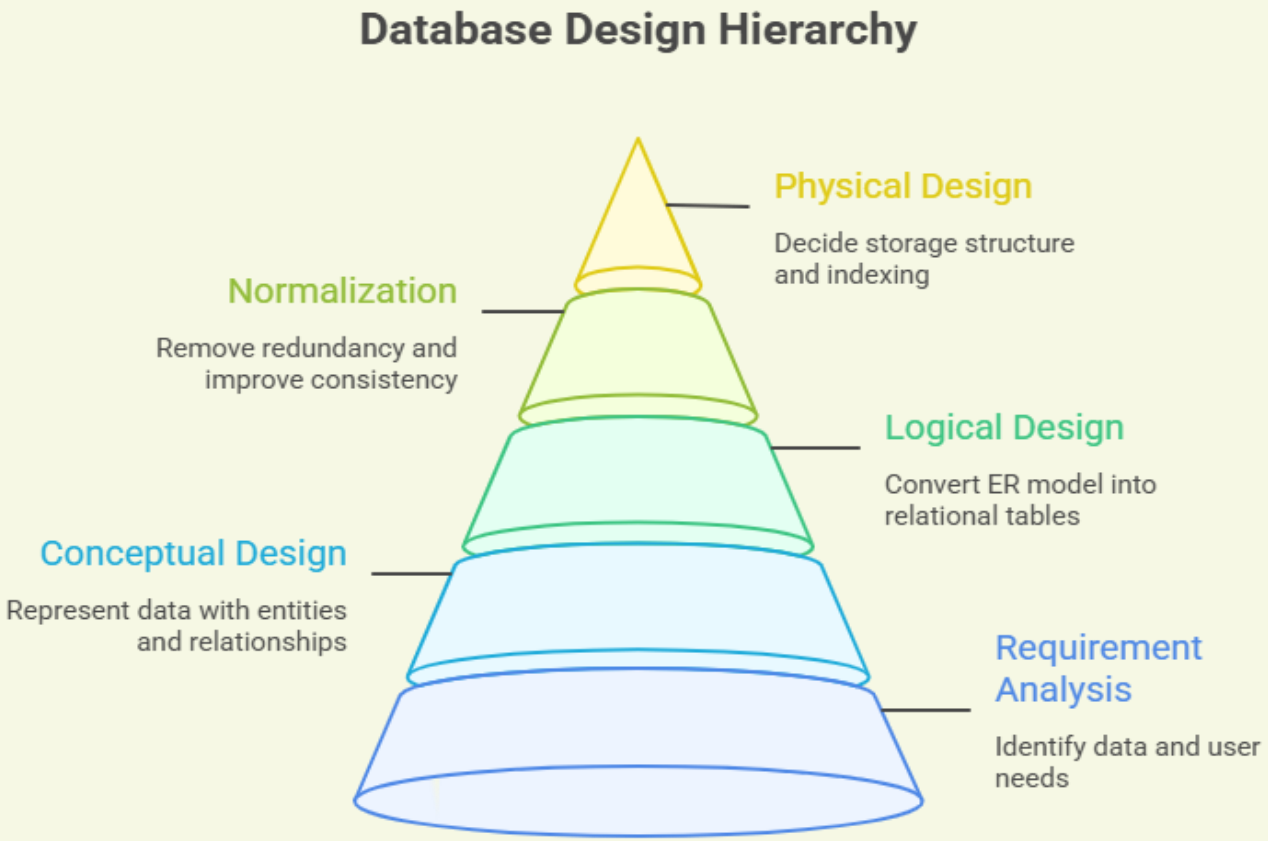
Database languages are used to **define, manipulate, control, and retrieve data** from a database.

Language	Purpose	Commands
DDL	Structure definition	CREATE, ALTER, DROP
DML	Data modification	INSERT, UPDATE, DELETE, SELECT
DCL	Access control	GRANT, REVOKE
TCL	Transaction control	COMMIT, ROLLBACK

SQL: Structures Query Language

Database Design

The process of designing the general structure of the database:



Made with Napkin

Data Models

Process Of Building A House

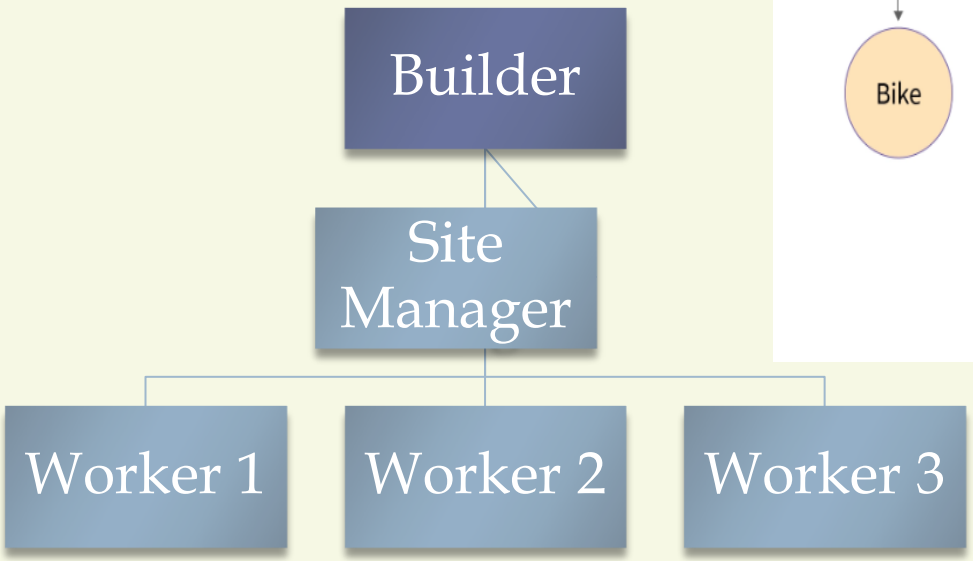
Data Models

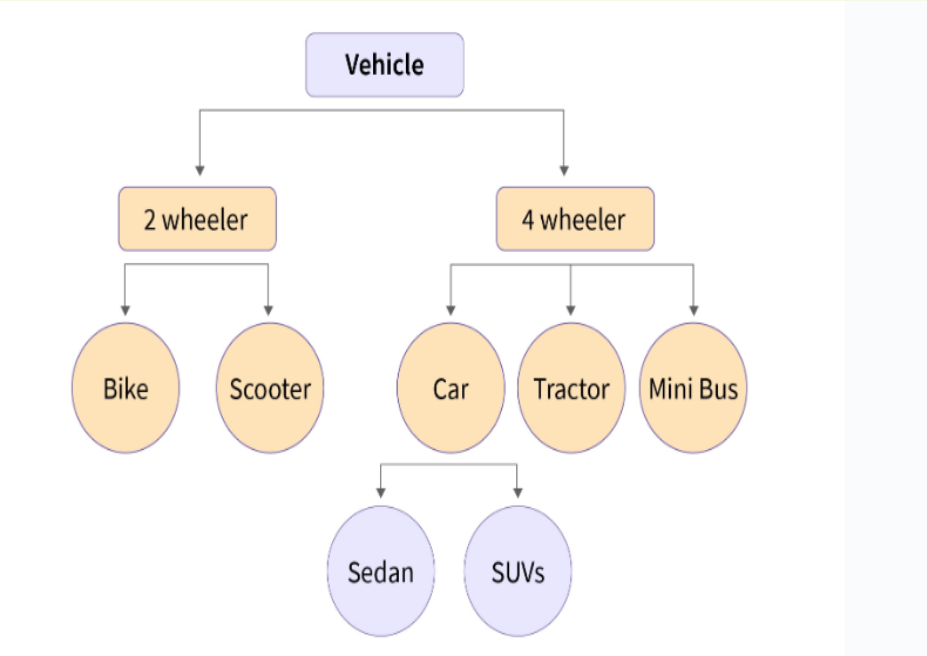
- A collection of tools for describing
  - Data
  - Data relationships
  - Data semantics
  - Data constraints

Types of Data Models

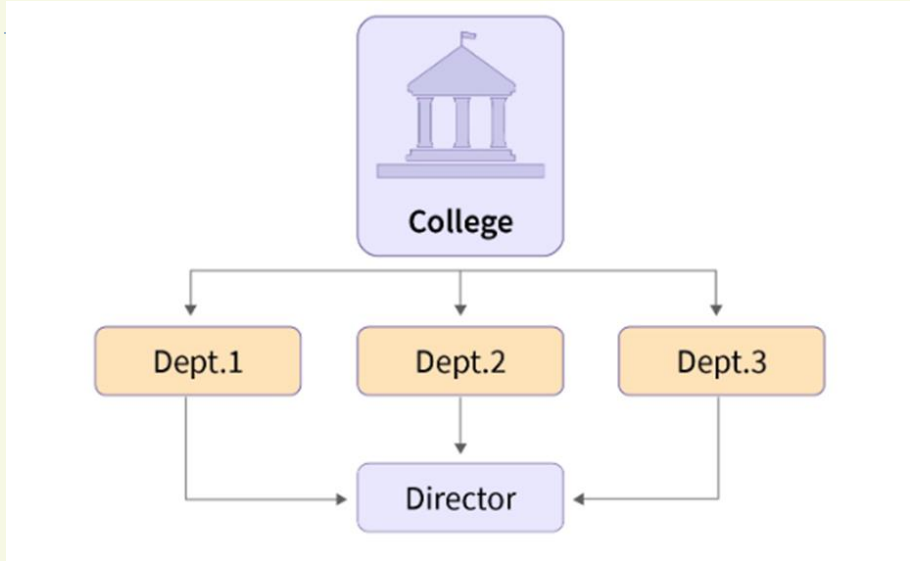
- Entity-relationship Model
- Network Model
- Hierarchical Model
- Relational Model
- Object Relation Model
- XML

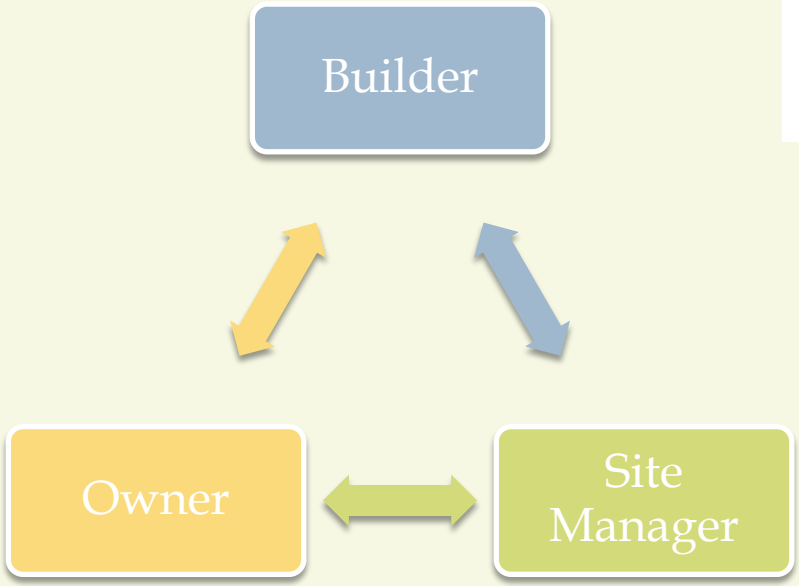
Hierarchical Model



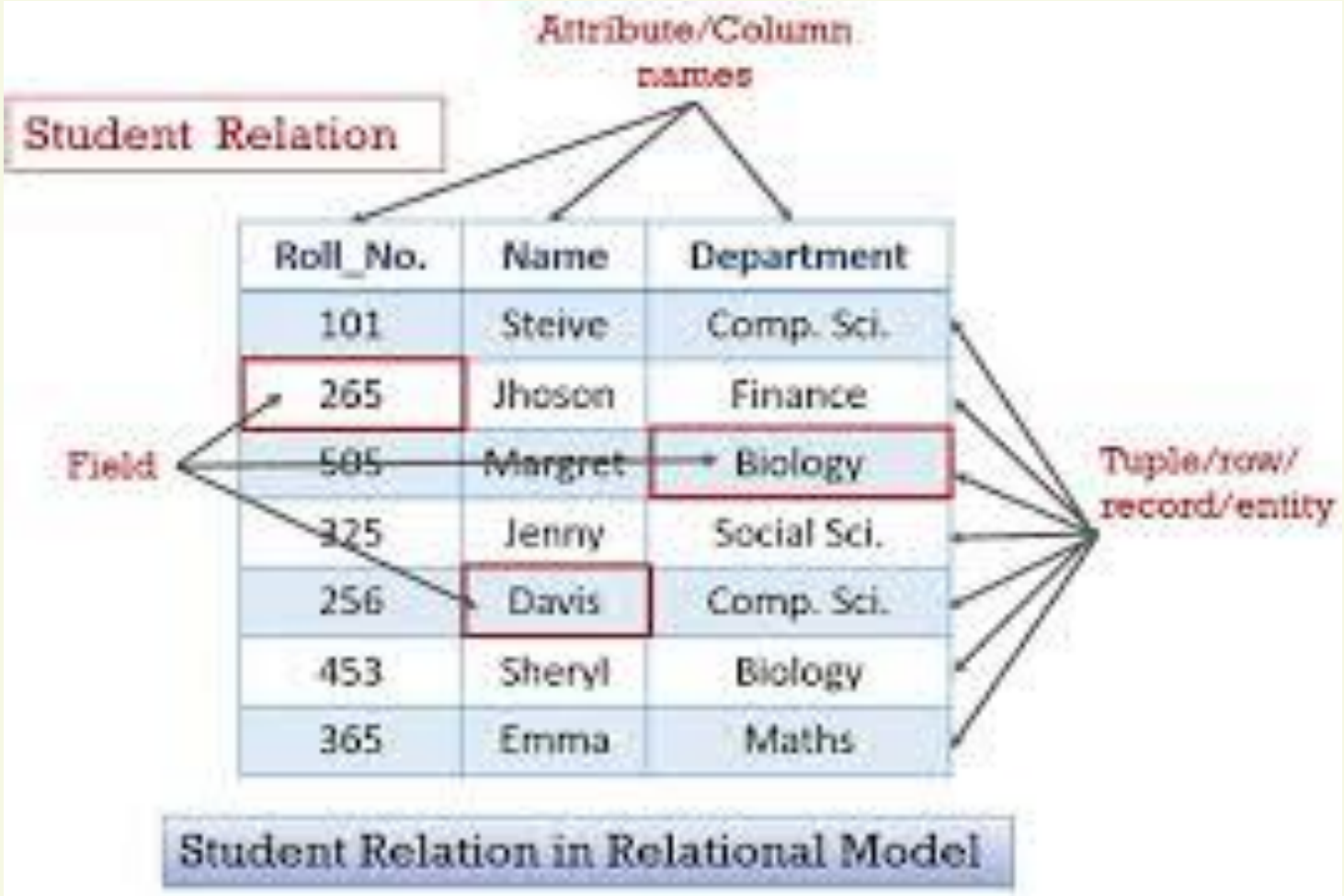


Network Model





Relational Model

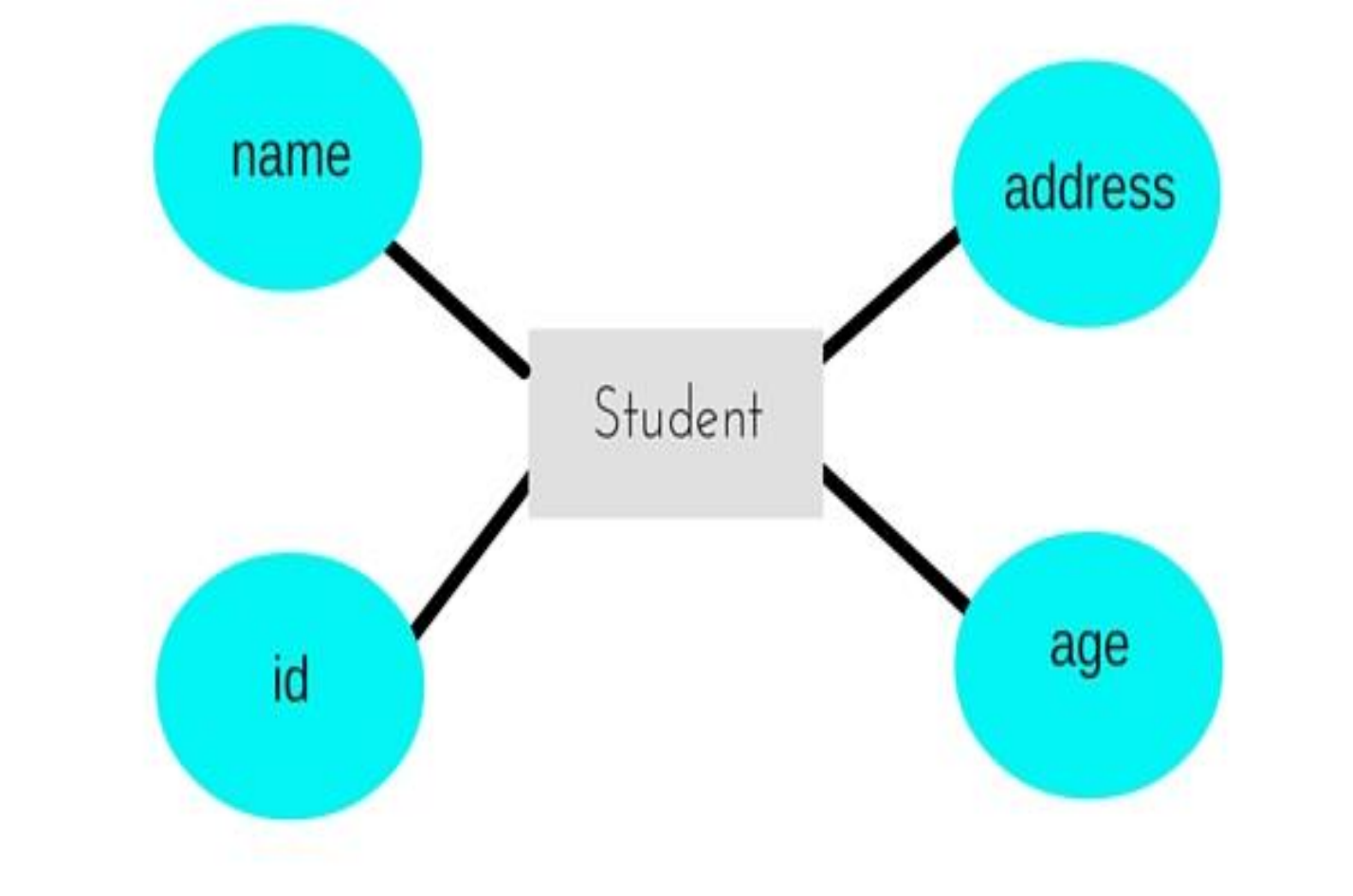


student_id	name	age
1	Akon	17
2	Bkon	18
3	Ckon	17
4	Dkon	18

subject_id	name	teacher
1	Java	Mr. J
2	C++	Miss C
3	C#	Mr. C Hash
4	Php	Mr. P H P

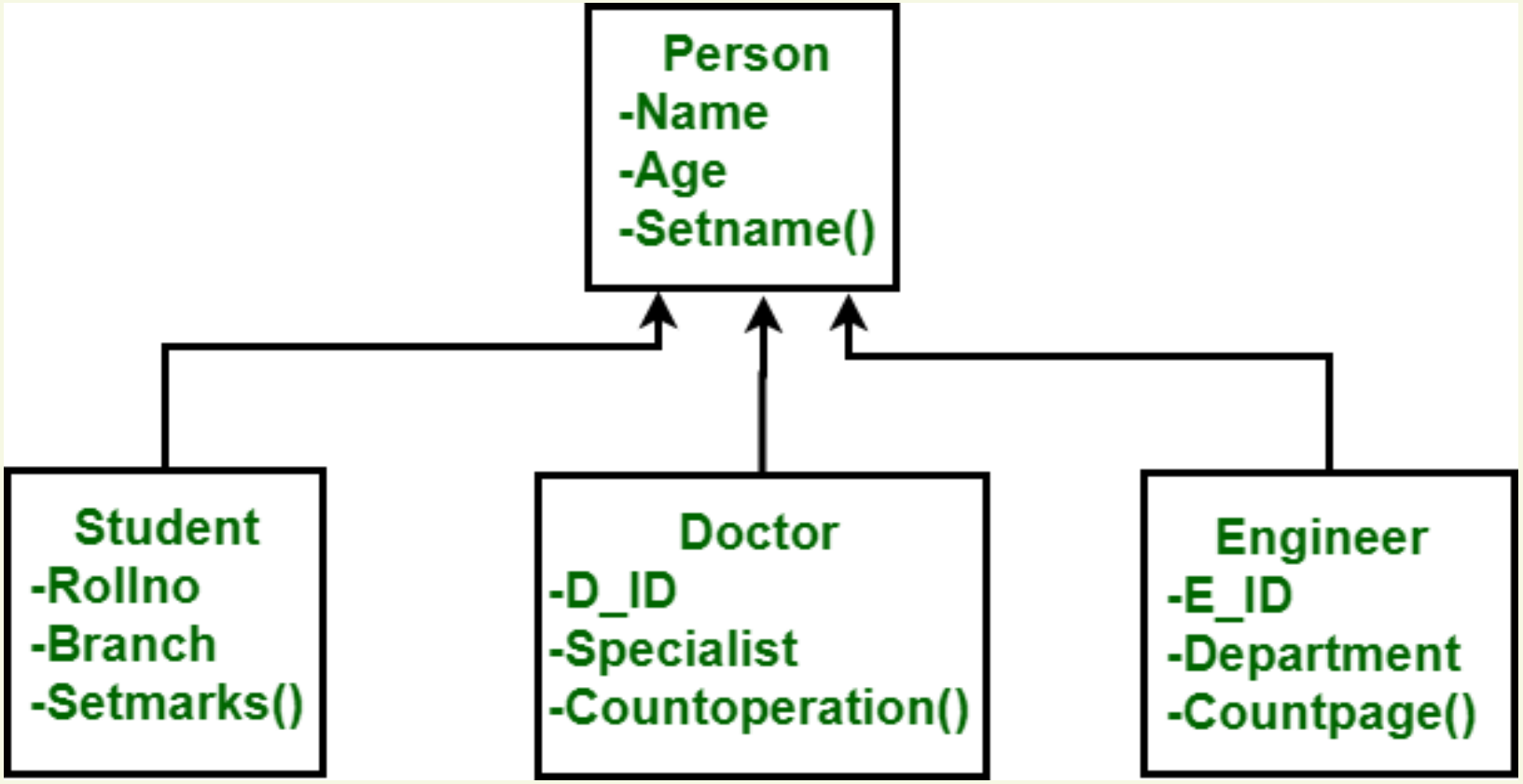
student_id	subject_id	marks
1	1	98
1	2	78
2	1	76
3	2	88

Entity-relationship Model



Object-Relational Data Models

- Relational model: flat, “atomic” values
- Object Relational Data Models
  - It combines the relational and Object-Oriented models, as the name implies. This model was developed to bridge the gap between the Object-Oriented and relational models. Many additional capabilities are available, such as the ability to create complicated data types based on our requirements utilizing existing data types


Person
-Name
-Age
-Setname()
Student
-Rollno
-Branch
-Setmarks()
Doctor
-D_ID
-Specialist
-Countoperation()
Engineer
-E_ID
-Department
-Countpage()

XML: Extensible Markup Language

- Defined by the WWW Consortium (W3C)
- Originally intended as a document markup language not a database language
- The ability to specify new tags, and to create nested tag structures made XML a great way to exchange **data**, not just documents
- XML has become the basis for all new generation data interchange formats.
- A wide variety of tools is available for parsing, browsing and querying XML documents/data

Constraints

- These are business rule that may affect the database and the data in it

Relational Constraints

- **Domain Constraint**
  Valid values for attributes
- **Tuple Uniqueness Constraint**
  No duplicate rows
- **Key Constraint**
  Unique identification
- **Entity Integrity Constraint**
  Primary key ≠ NULL
- **Referential Integrity Constraint**
  Valid relationship between tables

- **What distinguishes the Hierarchical model from the Network model in DBMS?**

1. The Hierarchical model allows many-to-many relationships, whereas the Network model only allows one-to-many.
2. The Network model allows many-to-many relationships, while the Hierarchical model is limited to one-to-many.
3. The Hierarchical model does not use any parent-child relationship.
4. The Network model is older than the Hierarchical model.

Quiz?

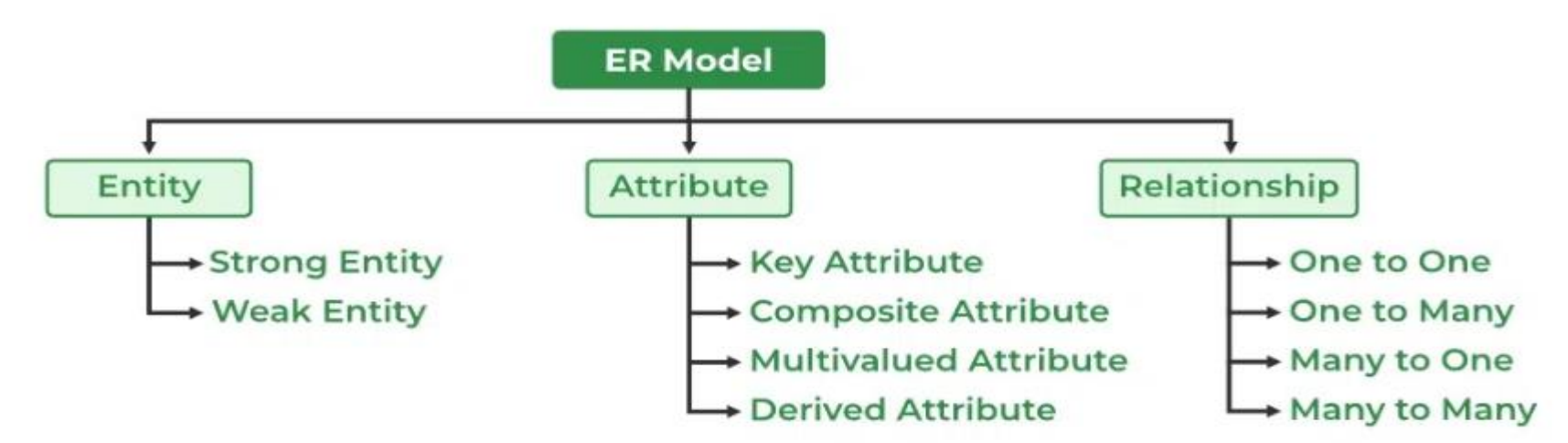
Which data model is described as providing a high-level conceptual view of the database and commonly visualized through ER diagrams?

1. Relational Model
2. Entity-Relationship Model (ER Model)
3. Object-Oriented Data Model
4. Associative Data Model

ER Model

ER Model

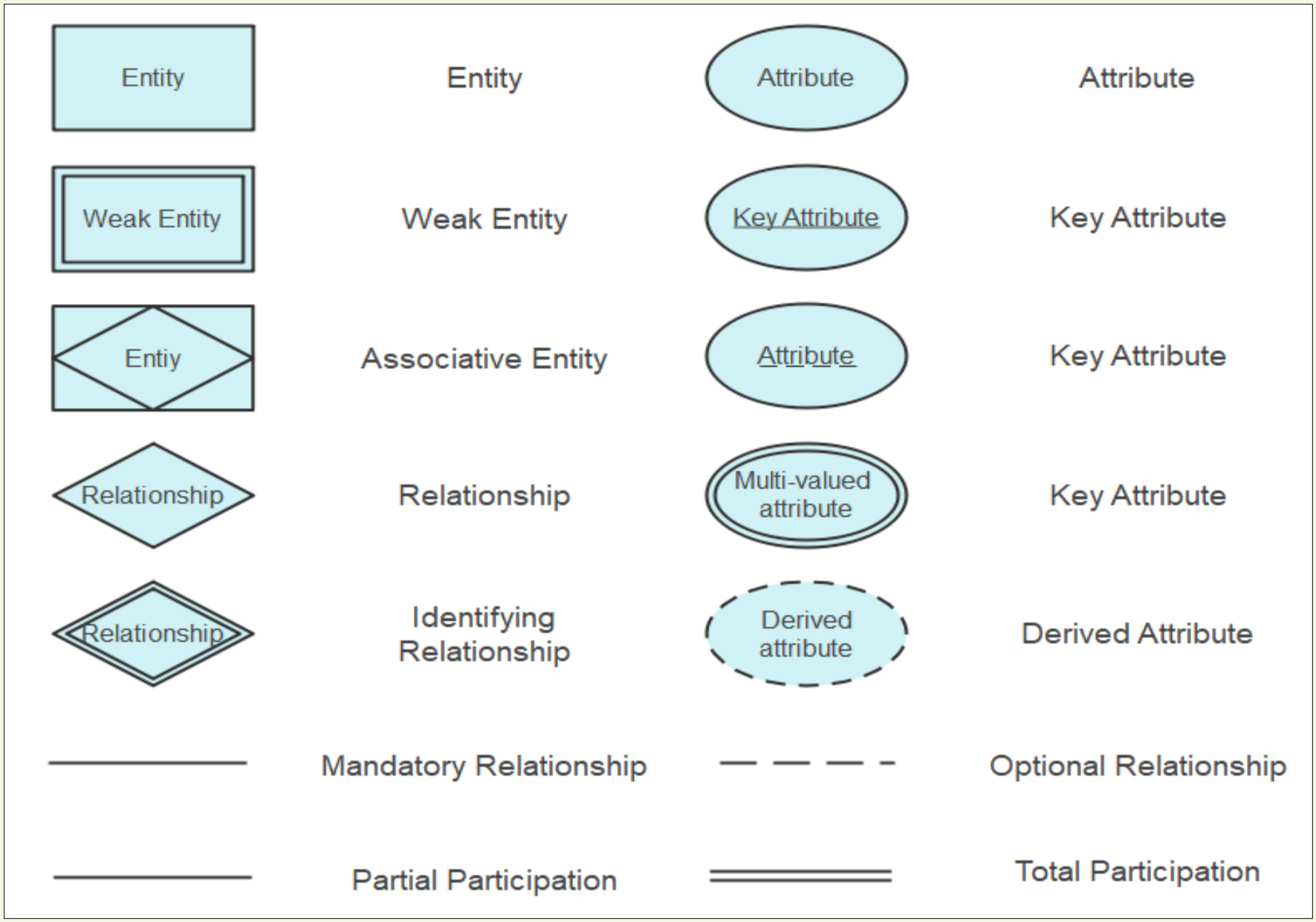
- **Conceptual model** for designing a database.
- Represent logical structure of a database, including entities, their attributes and relationships between them.
- **Entity:** An object that is stored as data, such as *Student*, *Course* or *Company*.
- **Attribute:** Properties that describe an entity, such as *StudentID*, *CourseName*, or *EmployeeEmail*.
- **Relationship:** A connection between entities, such as "a *Student* enrols in a *Course*".



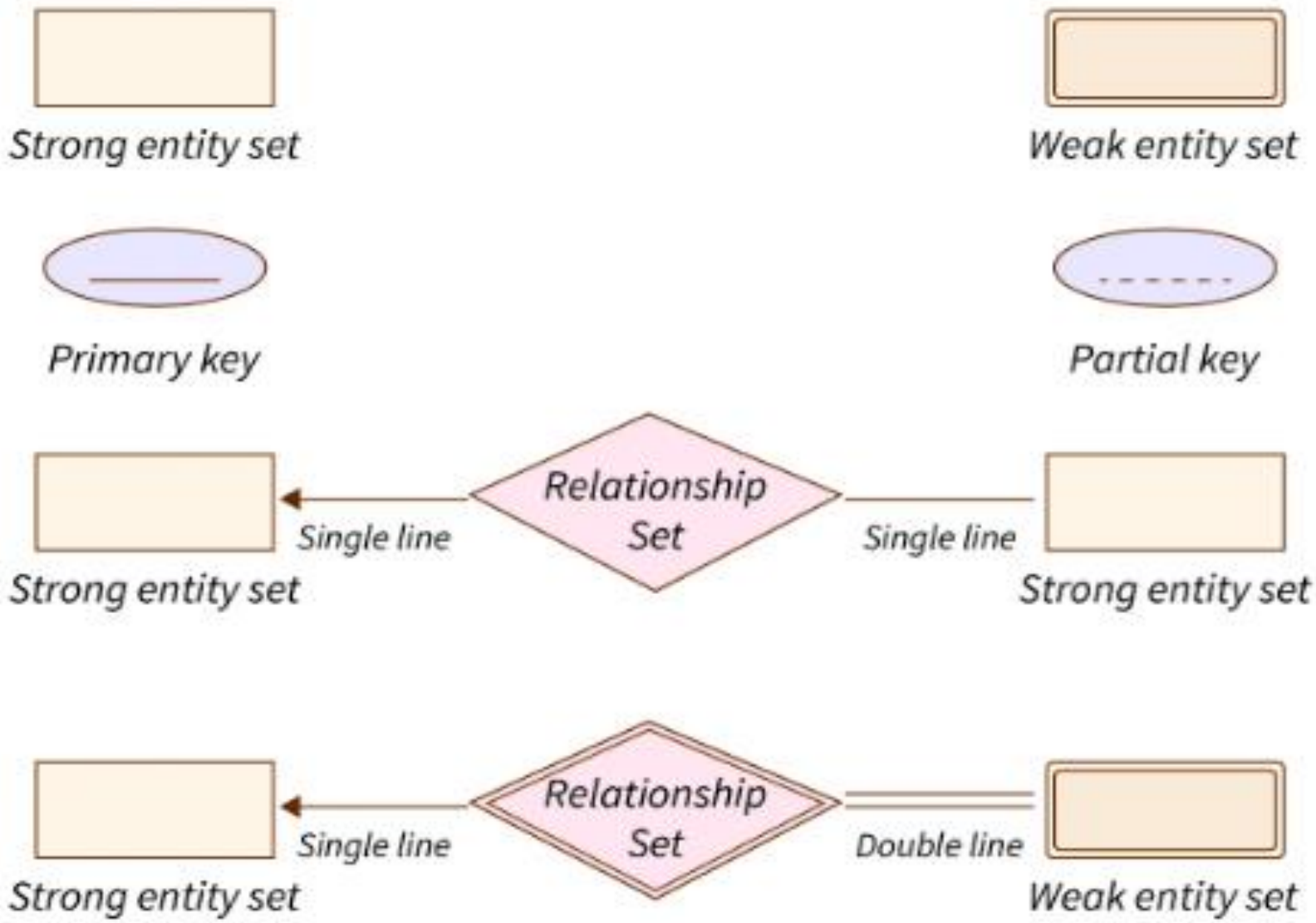
E-R diagram

E-R diagram

- ER diagrams represent the E-R model in a database, making them easy to convert into relations (tables).
- **ER Diagram** is the visual, graphical representation of that model, using symbols like boxes and diamonds to show those elements and connections.


Entity
Entity
Attribute
Attribute
Weak Entity
Weak Entity
Key Attribute
Key Attribute
Entiy
Associative Entity
Attribute
Key Attribute
Relationship
Relationship
Multi-valued attribute
Key Attribute
Relationship
Identifying Relationship
Derived attribute
Derived Attribute
Mandatory Relationship
Optional Relationship
Partial Participation
Total Participation

Symbols for ER Model



5 STEPS TO CREATE AN ER DIAGRAM

1. Define entities
What core concepts need to be stored in and represented by your diagram?

2. Establish relationships
How are your entities connected to one another?

3. Capture cardinality
How many different entities does each entity connect to?

4. Add attributes
What important details should you include about each entity?

5. Create and refine
Is labeling standardized? Are there any redundancies or inconsistencies?

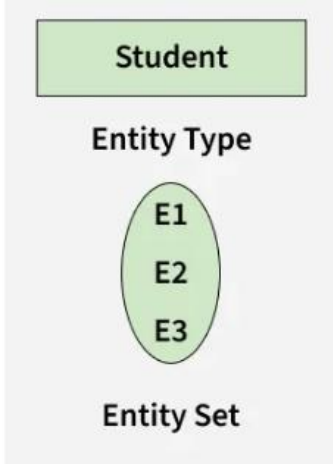
Entity

An Entity represents a real-world object, concept or thing about which data is stored in a database.
Tables in relational database represent these entities.
Example of entities:
Real-World Objects: Person, Car, Employee etc.
Concepts: Course, Event, Reservation etc.
Things: Product, Document, Device etc.
The entity type defines the structure of an entity, while individual instances of that type represent specific entities.

Entity : Entities are represented by means of rectangles.
Rectangles are named with the entity set they represent.

Entity Set

- An entity refers to an individual object of an entity type, and the collection of all entities of a particular type is called an entity set.
- For example, E1 is an entity that belongs to the entity type "Student," and the group of all students forms the entity set.
- In the ER diagram below, the entity type is represented as:



Types of Entity

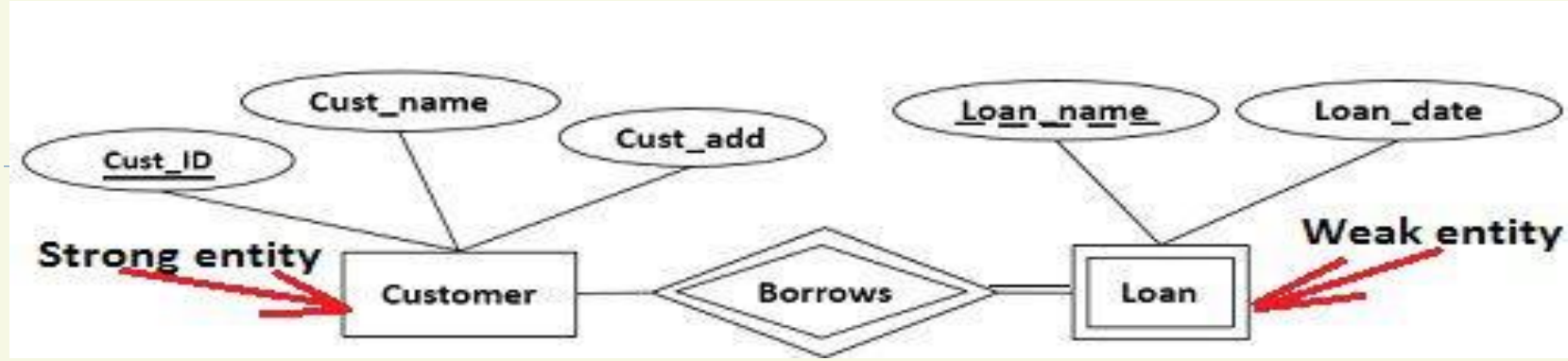
1. Strong Entity

- Entity has a key Attribute that can uniquely identify each instance of the entity.
- A Strong Entity does not depend on any other Entity in the Schema for its identification.
- It has a primary key that ensures its uniqueness and is represented by a rectangle in an ER diagram.

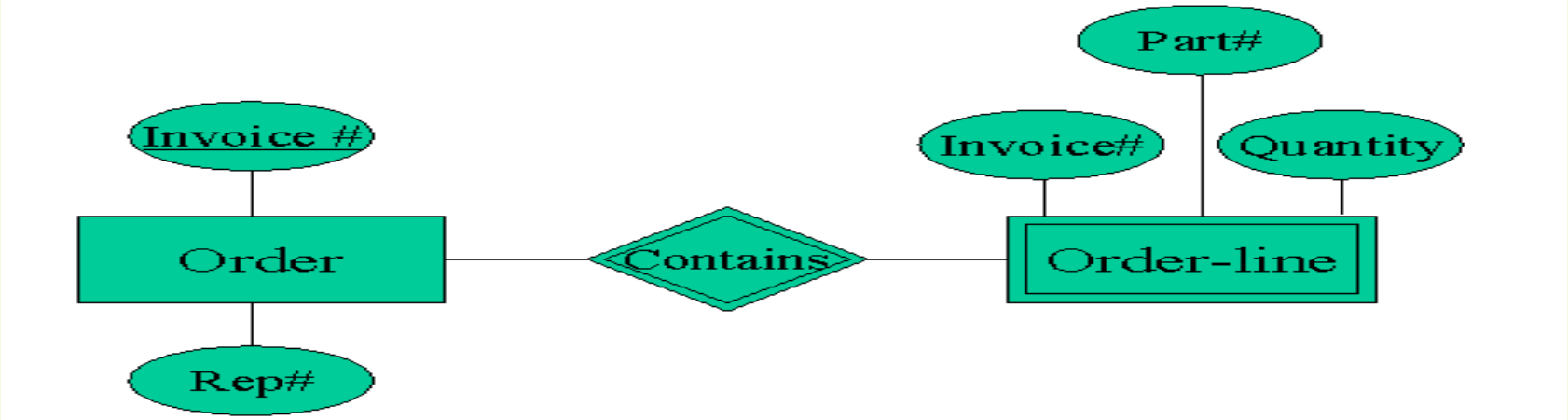
2. Weak Entity

- A Weak Entity cannot be uniquely identified by its own attributes alone.
- It depends on a strong entity to be identified. A
- weak entity is associated with an identifying entity (strong entity), which helps in its identification.
- A weak entity are represented by a double rectangle.

(The participation of weak entity types is always total. The relationship between the weak entity type and its identifying strong entity type is called identifying relationship and it is represented by a double diamond.)

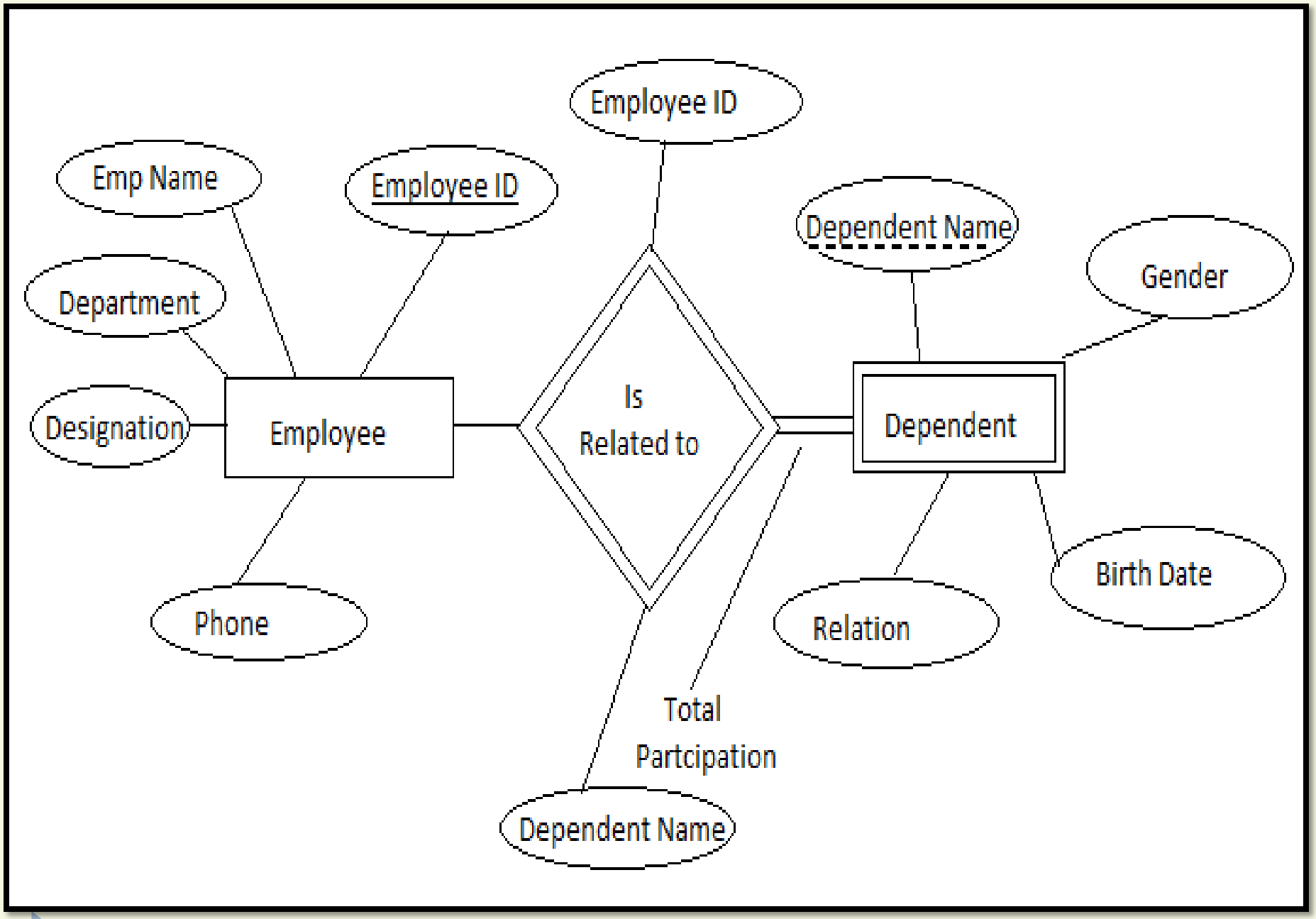

Cust_name
Cust_ID
Cust_add
Loan_name
Loan_date
Strong entity
Weak entity
Customer
Borrows
Loan

Student-id
Name
Student-id
amount
Students
Pay
Fees


Part#
Invoice #
Invoice#
Quantity
Order
Contains
Order-line
Rep#

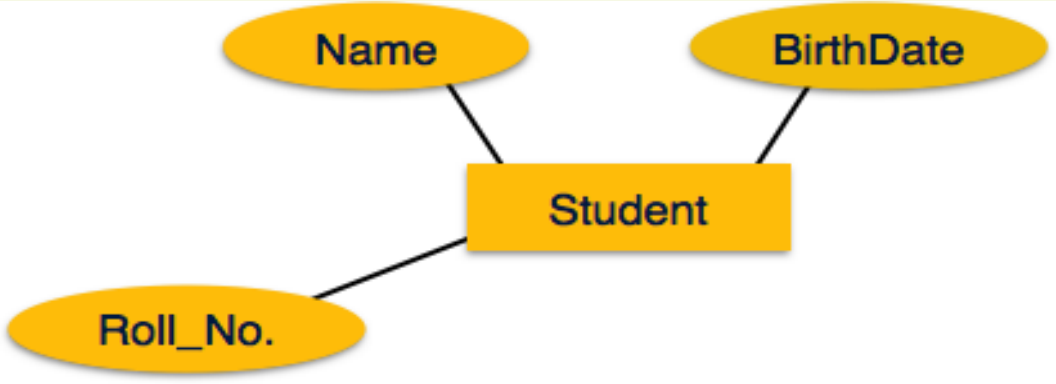
Comparison Between Strong and Weak Entity

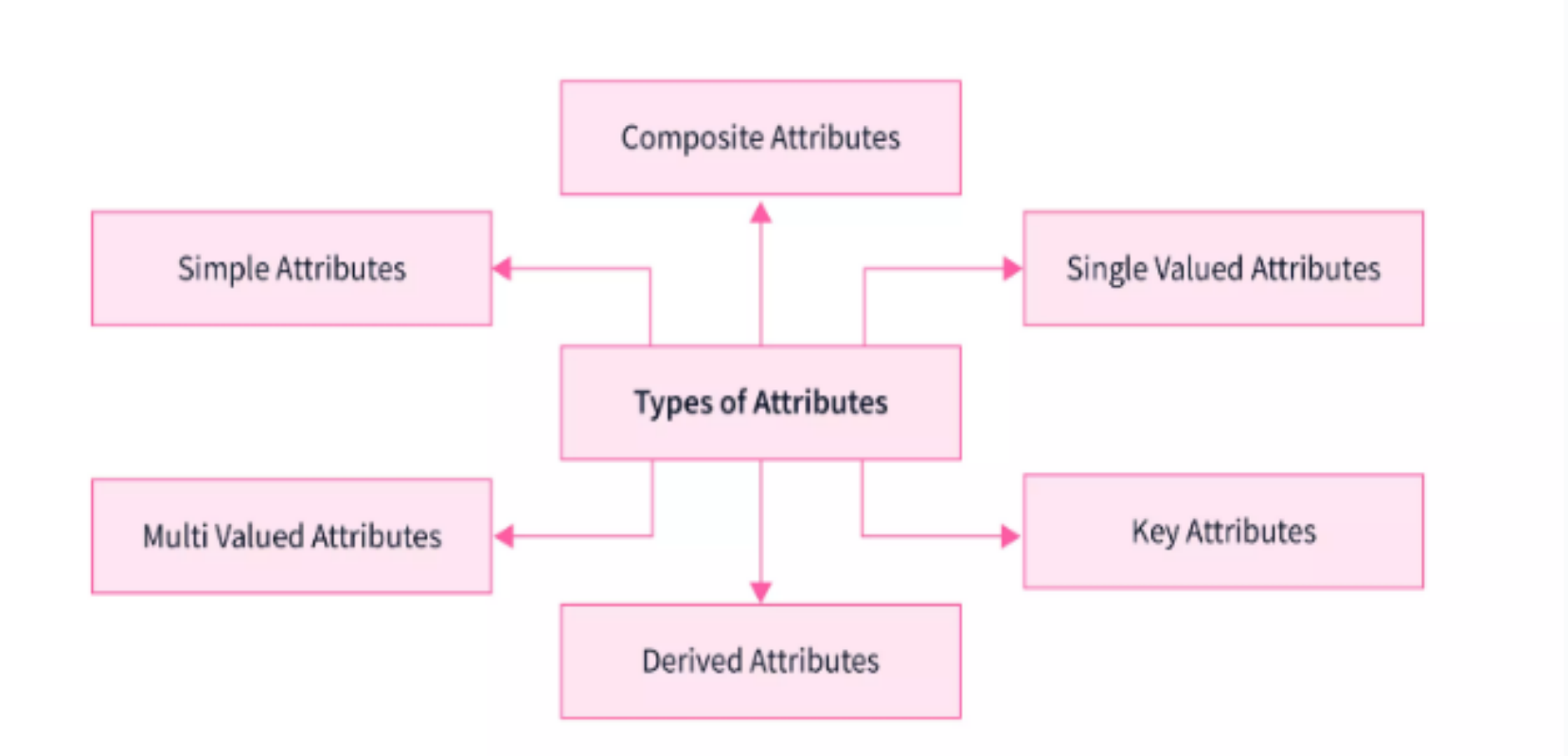
	Strong entity set	Weak entity set
Representation	To represent a strong entity set, a single rectangle is used.	To represent a weak entity set, a double rectangle is used.
Key attributes	A strong entity set always has attributes enough to select one as a primary key.	A weak entity set doesn't have a primary key. Instead, it has a partial key.
Dependence	The presence of a primary key makes a strong entity set independent.	As a weak entity set contains only a partial key, it is always dependent on a strong entity for its unique identification.
Relationship representation	A single diamond symbol is used to depict the relationship between two strong entities.	A double diamond symbol is used to represent the relationship between a strong and a weak entity set.
Participation constraint	Total participation may or may not be there by a strong entity set.	A weak entity set, being dependent on a strong entity set, always has its total participation in the relationship between the entities.
Relationship connection	A single line is used to establish connection with the relationship set.	A double line is used to establish connection with the relationship set.


Employee ID
Emp Name
Employee ID
Dependent Name
Gender
Department
Is
Related to
Designation
Employee
Dependent
Birth Date
Phone
Relation
Total
Partcipation
Dependent Name

Attributes

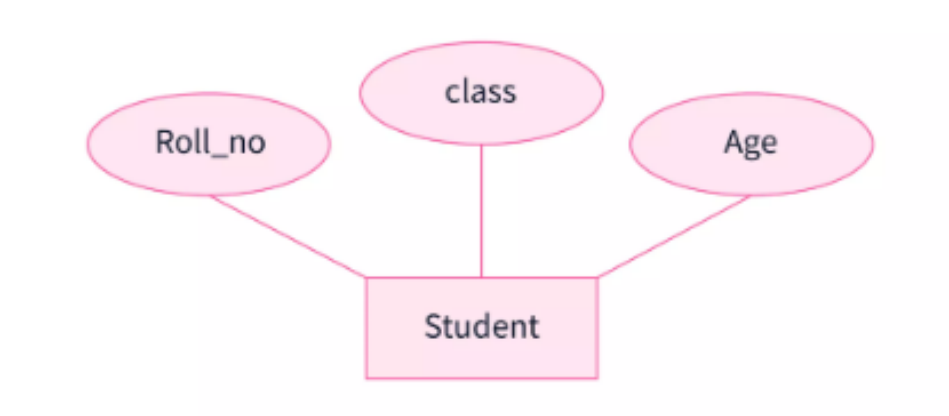
- Attributes are the properties of entities.
- Attributes are represented by means of ellipses. Every ellipse represents one attribute and is directly connected to its entity (rectangle).
- attributes provide additional information and describe the characteristics & properties of an entity in DBMS.
- An entity may contain any number of attributes while one of the attributes is considered to be a primary key attribute.




Composite Attributes
Simple Attributes
Single Valued Attributes
Types of Attributes
Multi Valued Attributes
Key Attributes
Derived Attributes

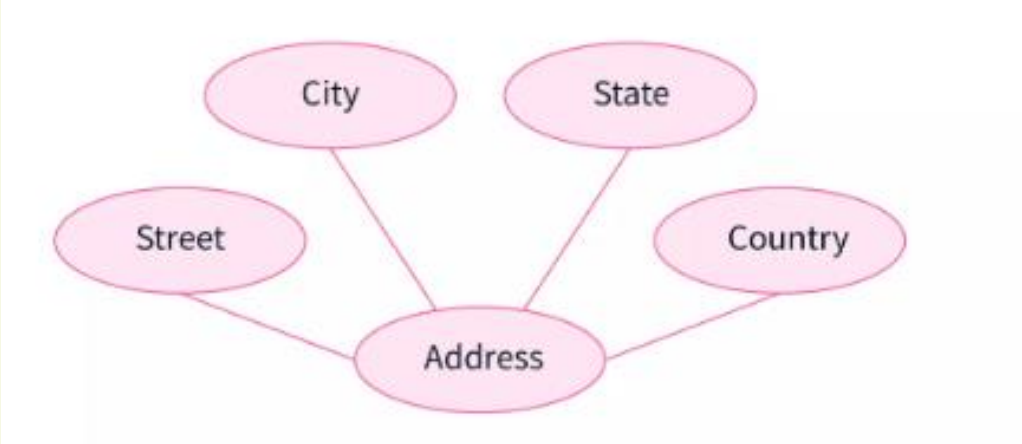
Simple Attributes

- Simple attributes in an ER model diagram are independent attributes that can't be classified further and also, can't be subdivided into any other component.
- These attributes are also known as **atomic attributes.**



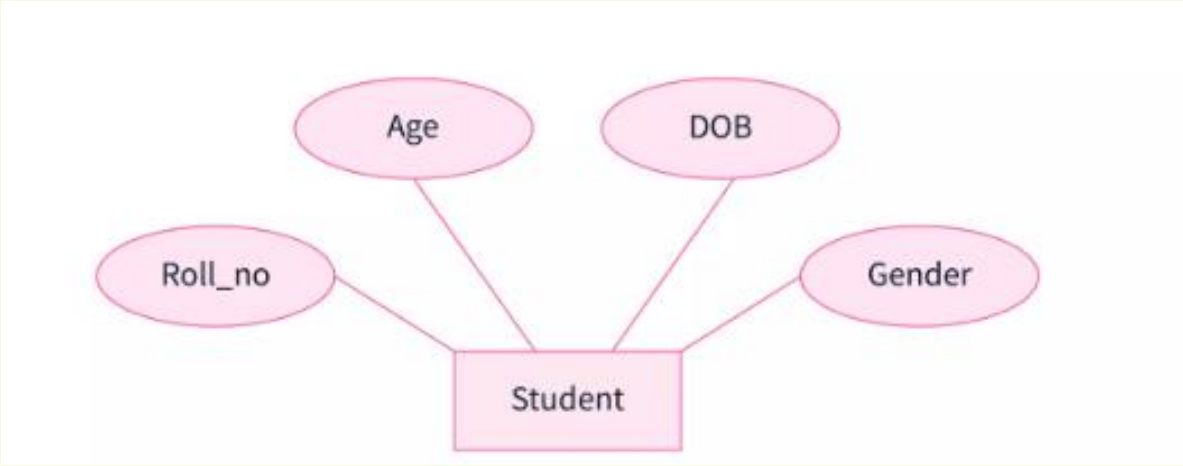
Composite Attributes

- These attributes can further subdivide composite attributes into different components or sub-parts that form simple attributes.
- In simple terms, composite attributes are composed of one or more simple attributes.



Single-Valued Attributes

- These consist of a single value for each entity instance and can't store more than one value.
- The value of these single-valued attributes always remains the same, just like the name of a person.



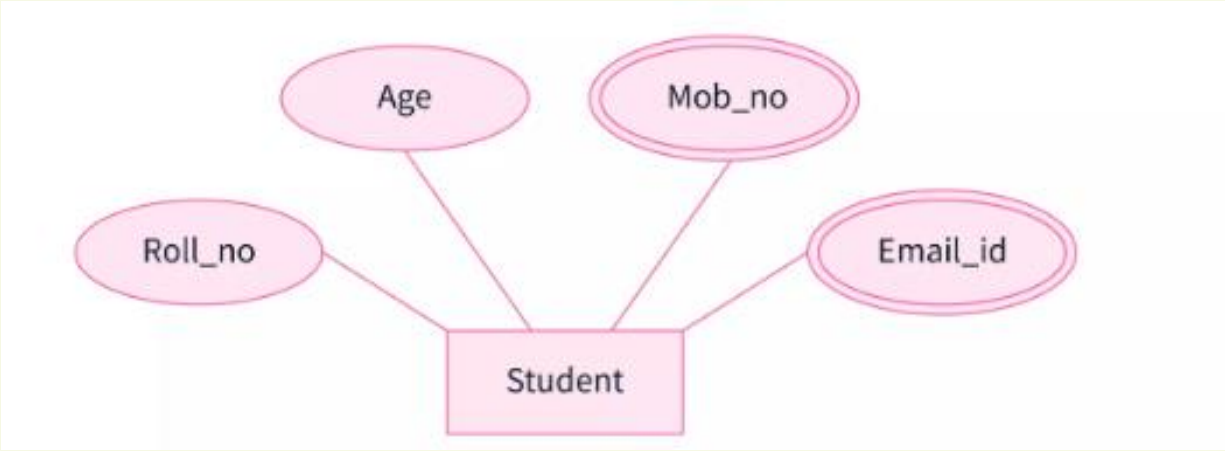
Quiz

- ***Which of the following statements accurately describes single-valued attributes in a DBMS?***

1. *Single-valued attributes cannot be further subdivided into other components.*
2. *Single-valued attributes can store multiple values for each entity instance.*
3. *Single-valued attributes are always represented by a co-centric elliptical shape.*
4. *Single-valued attributes are rarely used in database modeling.*

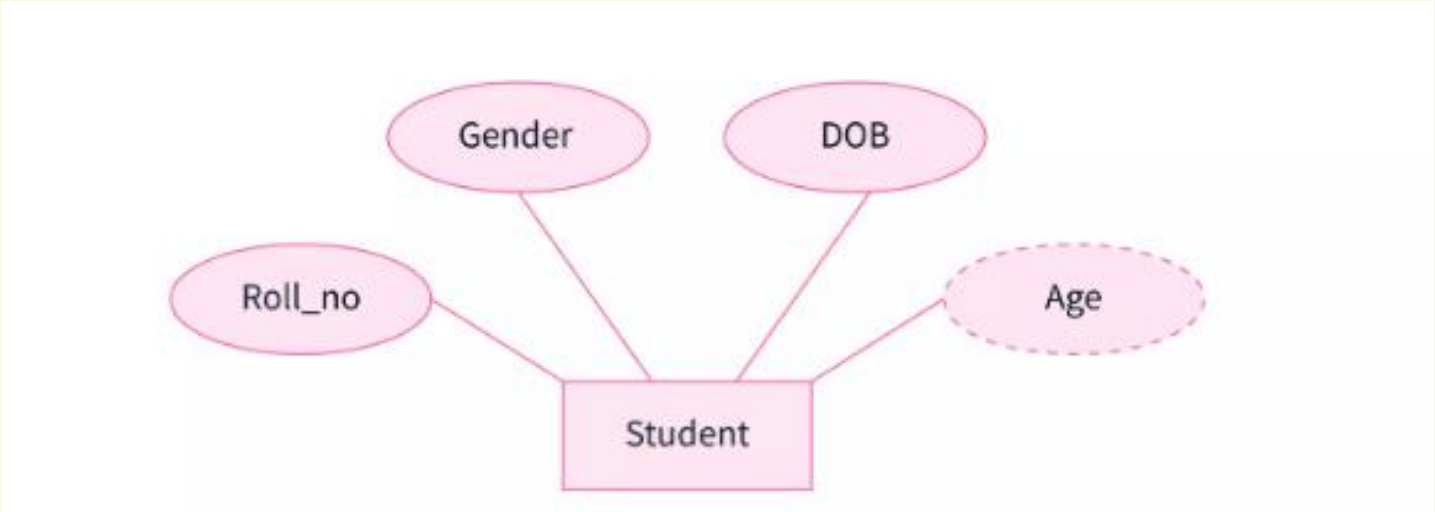
Multi-Valued Attributes

- Multi-valued attributes can take up and store more than one value at a time for an entity instance from a set of possible values.
- These attributes are represented by **co-centric elliptical shape**, and we can also use **curly braces { }** to represent multi-valued attributes inside it.



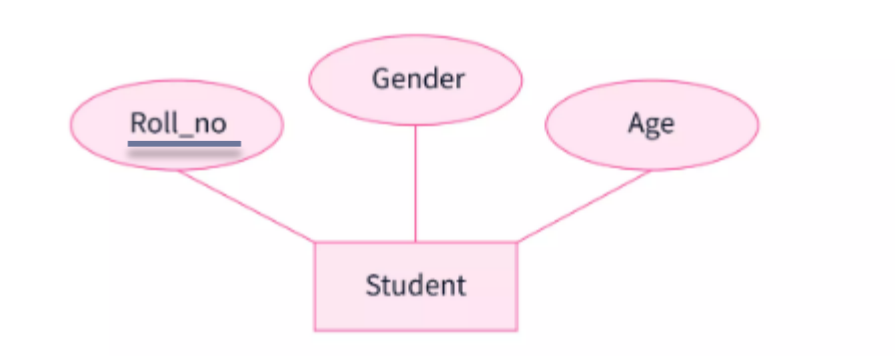
Derived Attributes

- Derived attributes in DBMS are those attributes whose values can be derived from the values of other attributes.
- They are always dependent upon other attributes for their value.
- **Derived attributes are always represented by dashed or dotted elliptical shapes.**



Key Attributes

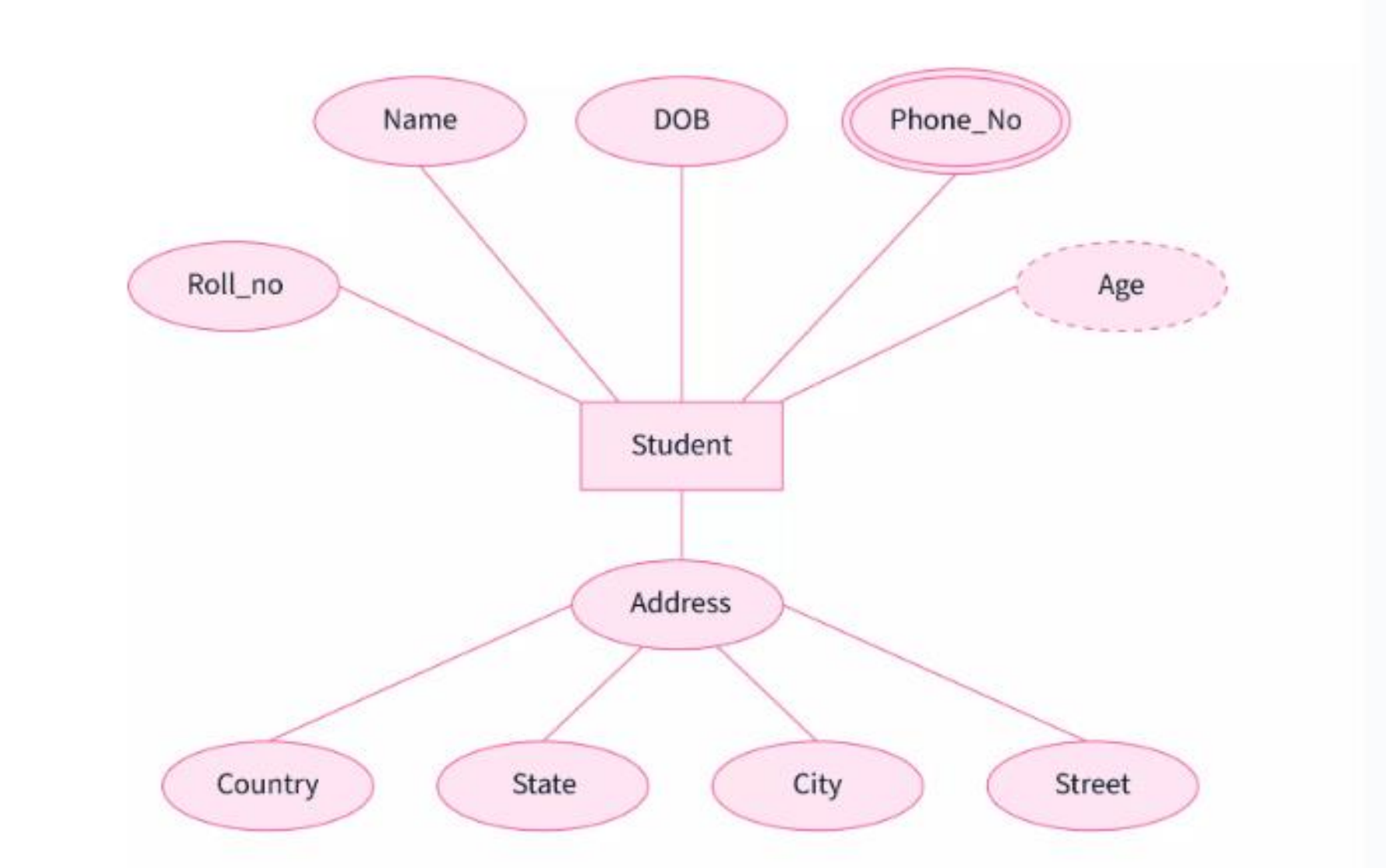
- Key attributes are special types of attributes that act as the primary key for an entity and they can uniquely identify an entity from an entity set.
- The values that key attributes store must be unique and non-repeating.



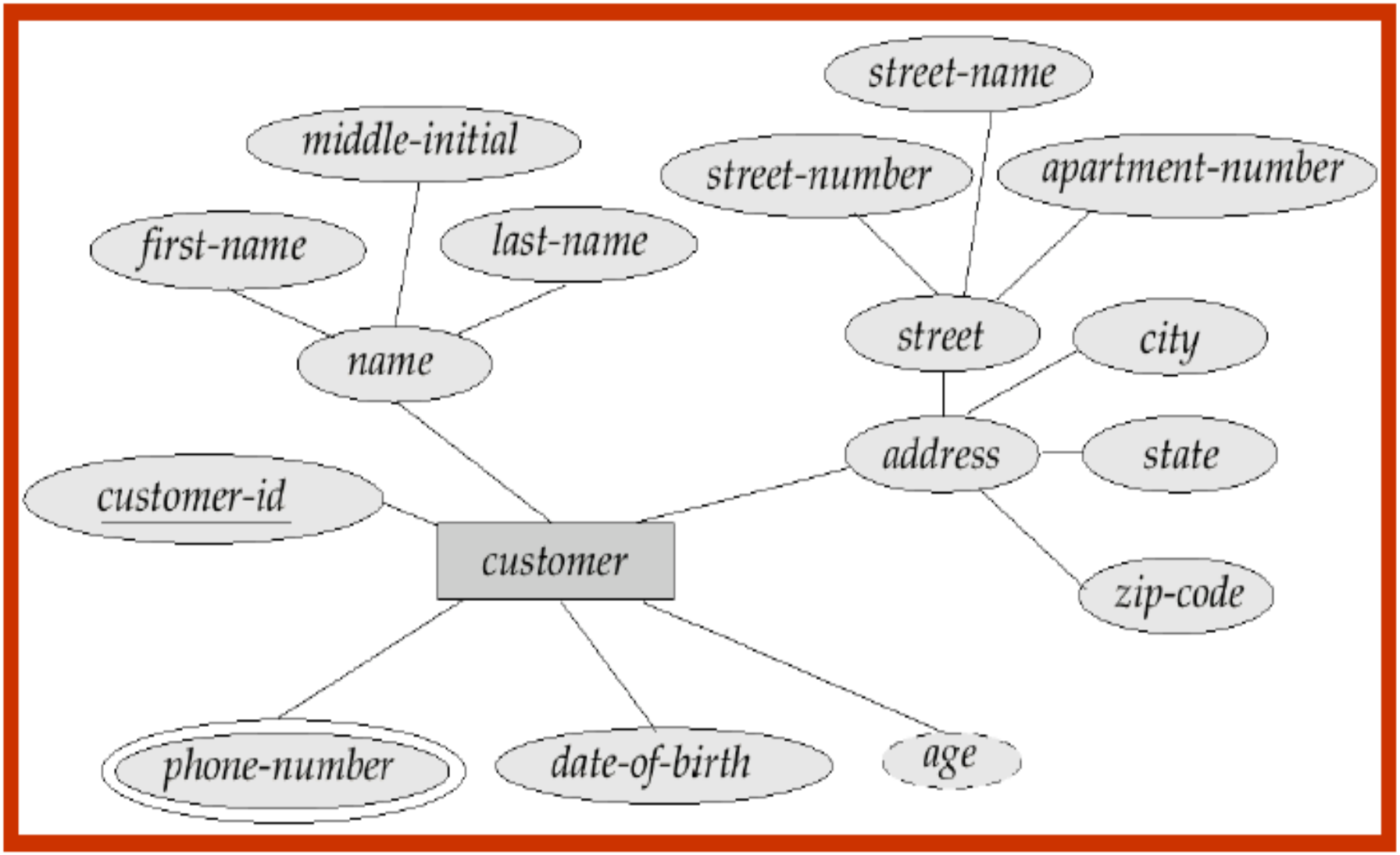
Quiz

- **What defines derived attributes in a database management system?**

1. *Derived attributes are independent and do not rely on other attributes for their values.*
2. *Derived attributes can store multiple values for each entity instance.*
3. *Derived attributes are represented by a solid elliptical shape in the ER model.*
4. *Derived attributes derive their values from other attributes within the database.*


Name
DOB
Phone_No
Roll_no
Age
Student
Address
Country
State
City
Street

E-R Diagram With Composite, Multivalued, and Derived Attributes



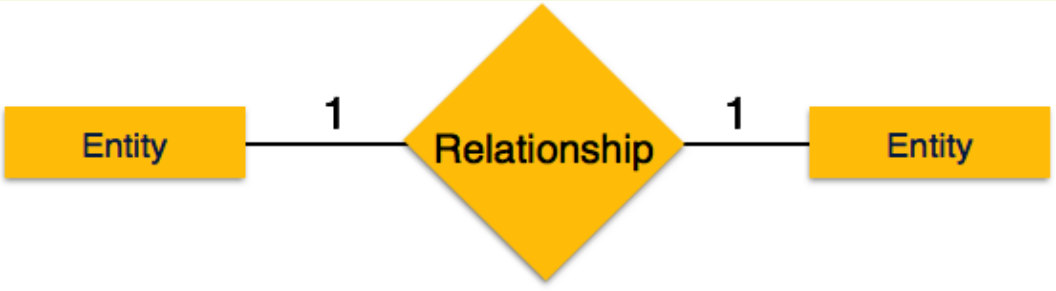
Relationship

Relationships are represented by diamond-shaped box. Name of the relationship is written inside the diamond-box.

All the entities (rectangles) participating in a relationship.

Binary Relationship and Cardinality

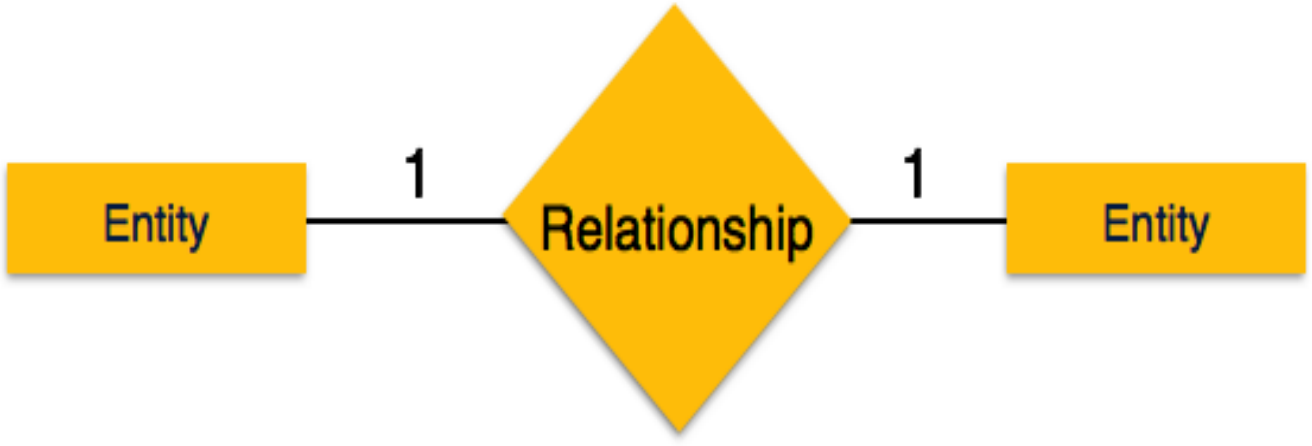
- A relationship where two entities are participating is called a **binary relationship**.
- **Cardinality** is the number of instance of an entity from a relation that can be associated with the relation.



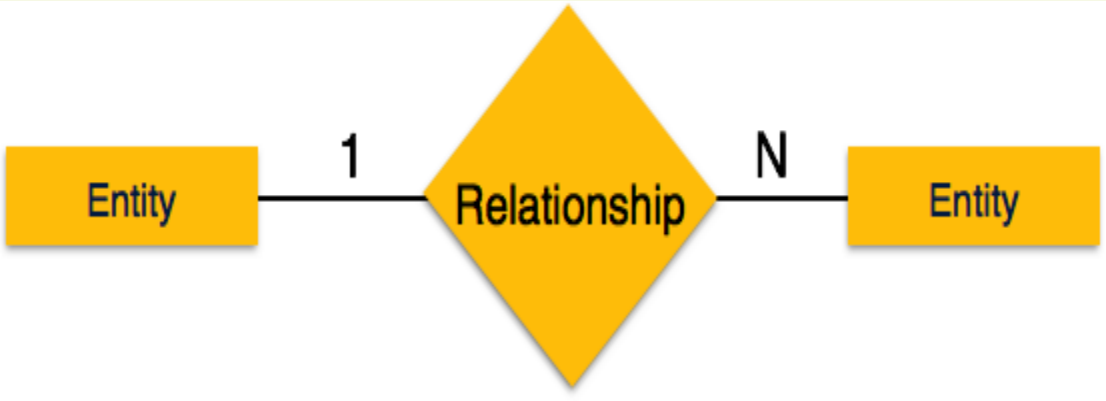
Mapping Cardinality

- One to One
- One to Many
- Many to One
- Many to Many

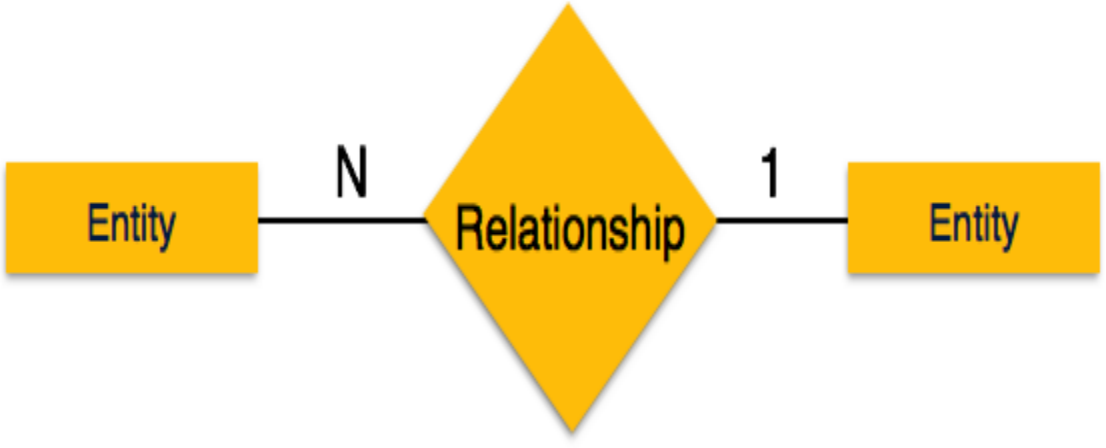
One-to-one − One entity from entity set A can be associated with at most one entity of entity set B and vice versa.



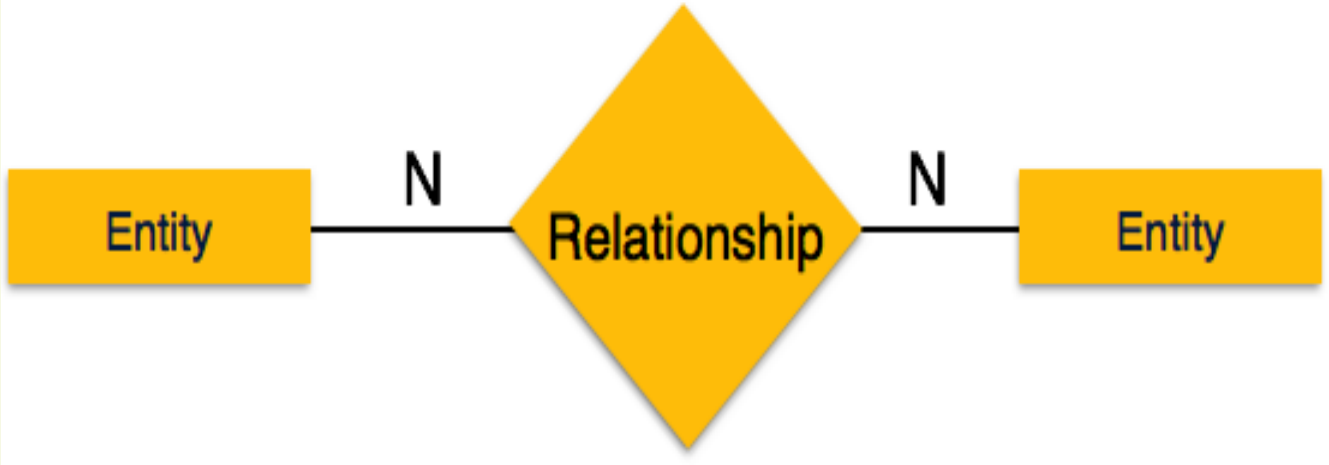
One-to-many − One entity from entity set A can be associated with more than one entities of entity set B however an entity from entity set B, can be associated with at most one entity.



Many-to-one − More than one entities from entity set A can be associated with at most one entity of entity set B, however an entity from entity set B can be associated with more than one entity from entity set A.

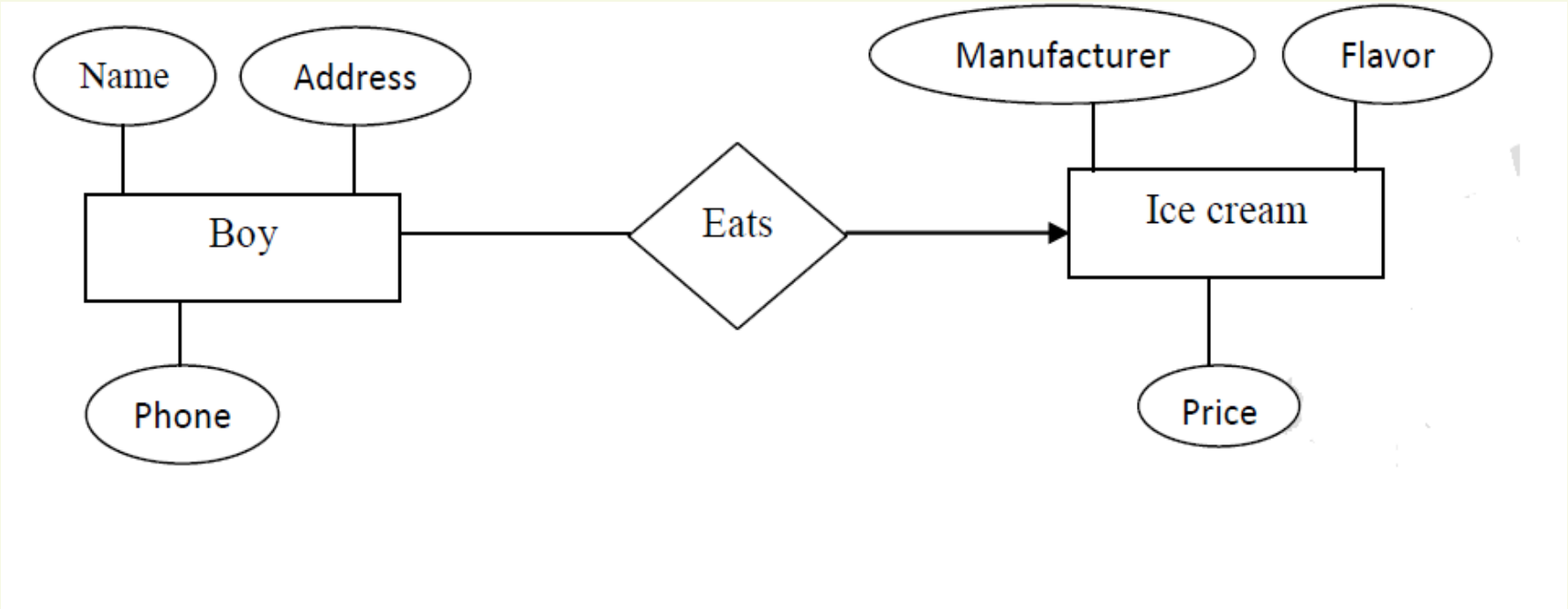


Many-to-many − One entity from A can be associated with more than one entity from B and vice versa.



How to Draw a ER Diagram

- A boy eats an ice-cream
- Entities are Boy and Ice- cream



Entity Relationship (ER) model for a college database

1. A college contains many departments
2. Each department can offer any number of courses
3. Many instructors can work in a department
4. An instructor can work only in one department
5. For each department there is a Head
6. An instructor can be head of only one department
7. Each instructor can take any number of courses
8. A course can be taken by only one instructor
9. A student can enroll for any number of courses
10. Each course can have any number of students

Step 1 : Identify the Entities

- Department
- Course
- Instructor
- Student
- College

Step 3: Identify the key attributes

"Department_Name" can identify a department uniquely.

Hence Department_Name is the key attribute for the Entity "Department".

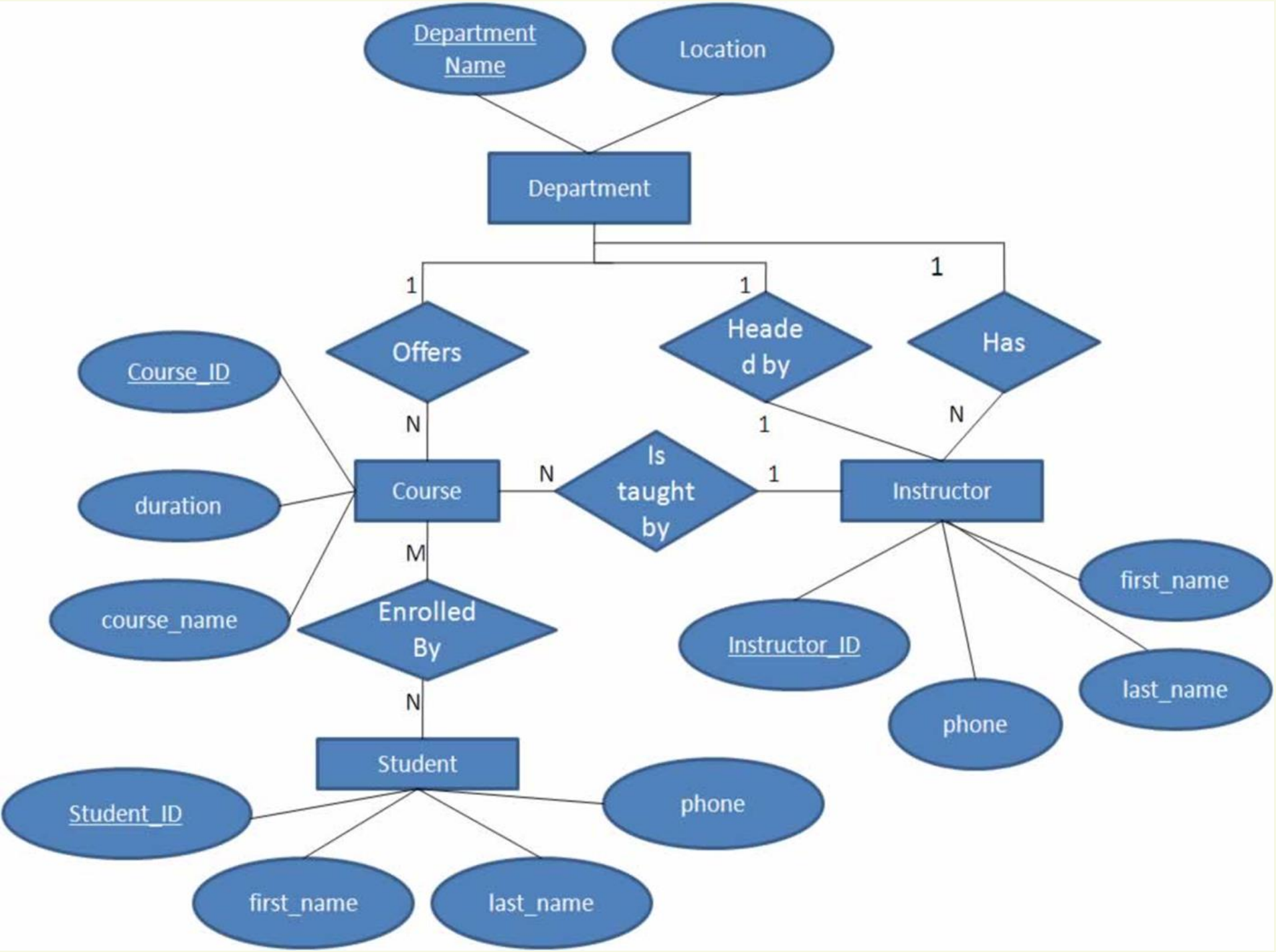
Course_ID is the key attribute for "Course" Entity.

Student_ID is the key attribute for "Student" Entity.

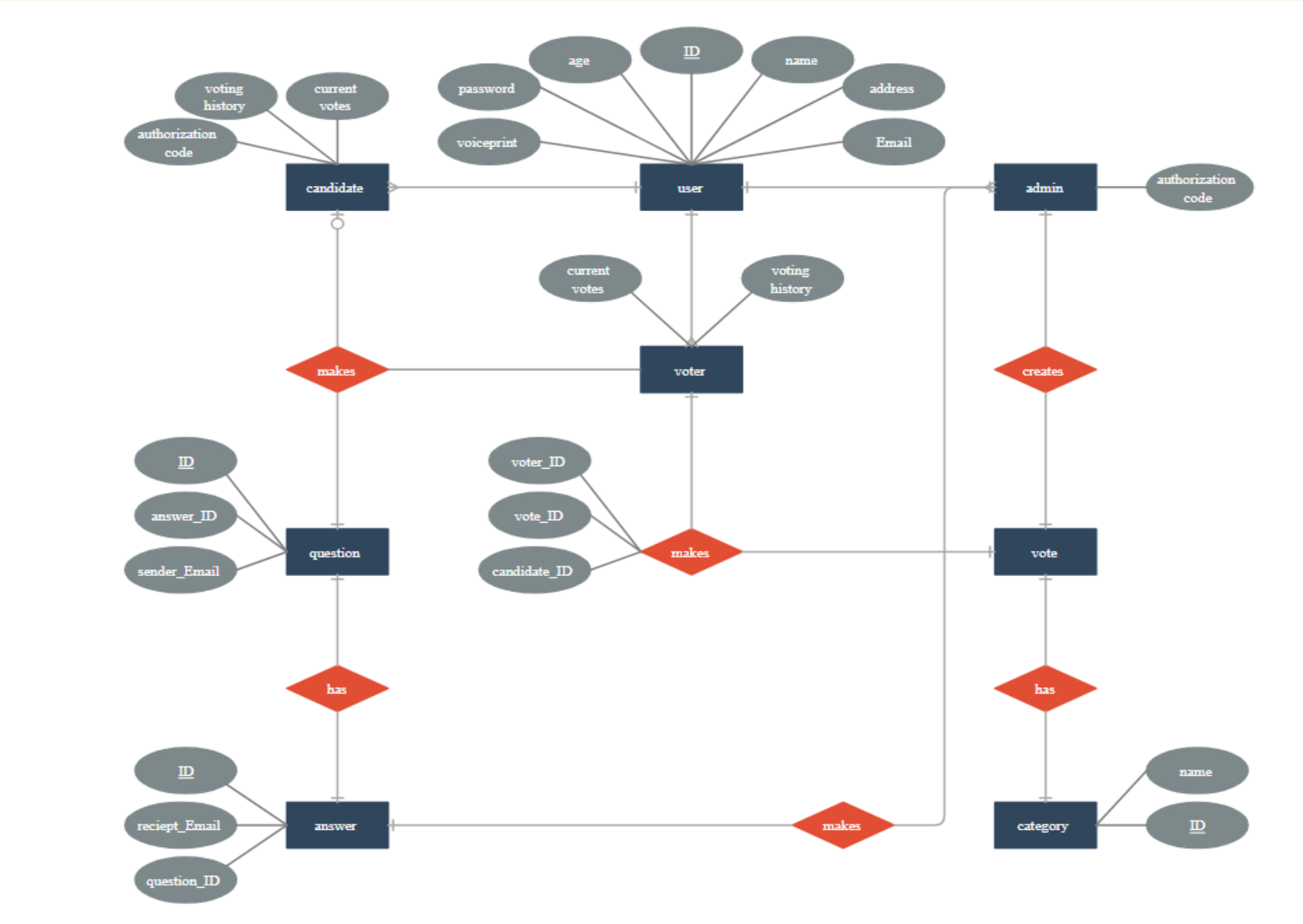
Instructor_ID is the key attribute for "Instructor" Entity.

Step 4: Identify other relevant attributes

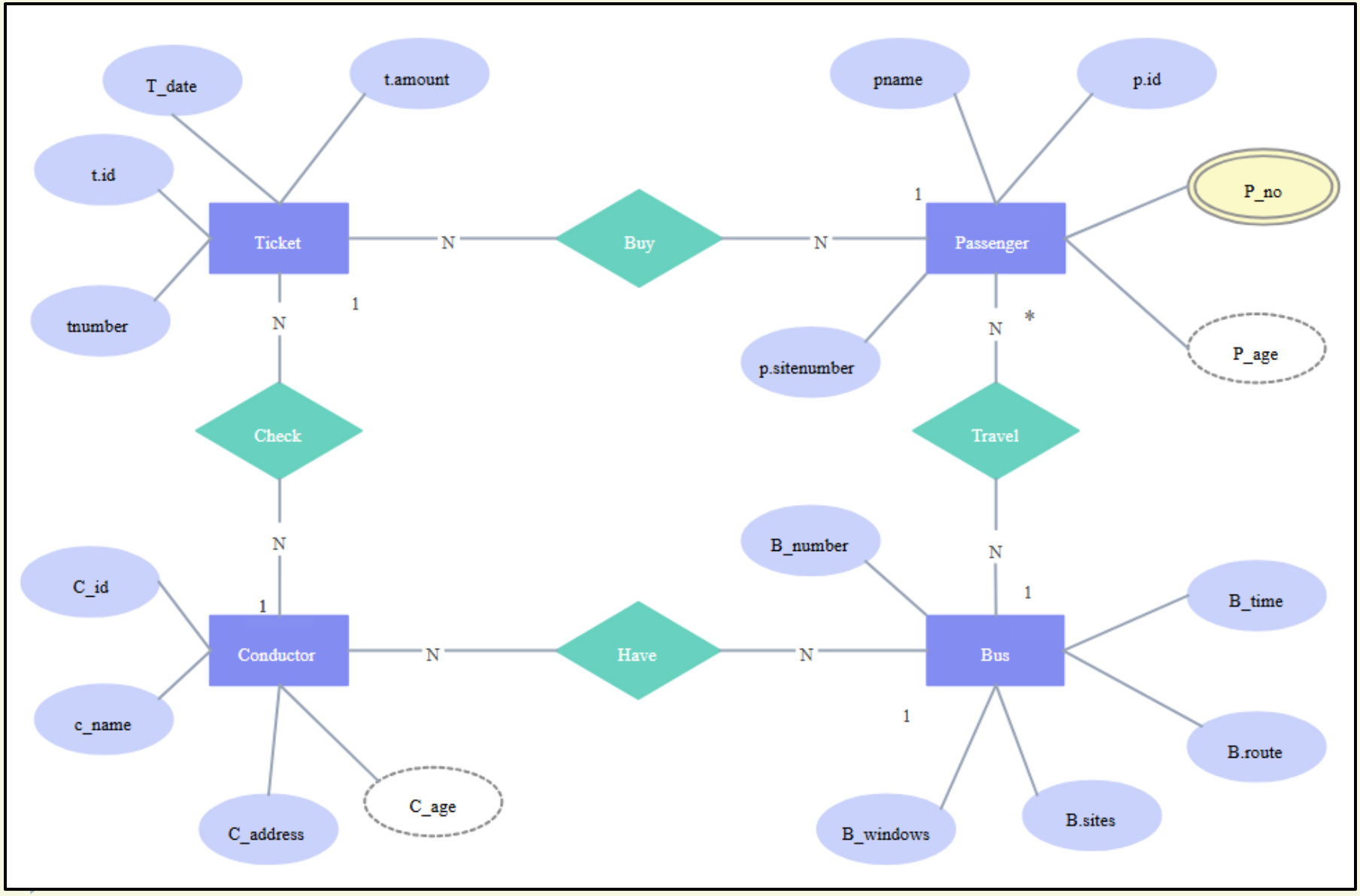
- For the department entity, other attributes are location
- For course entity, other attributes are course_name, duration
- For instructor entity, other attributes are first_name, last_name, phone
- For student entity, first_name, last_name, phone

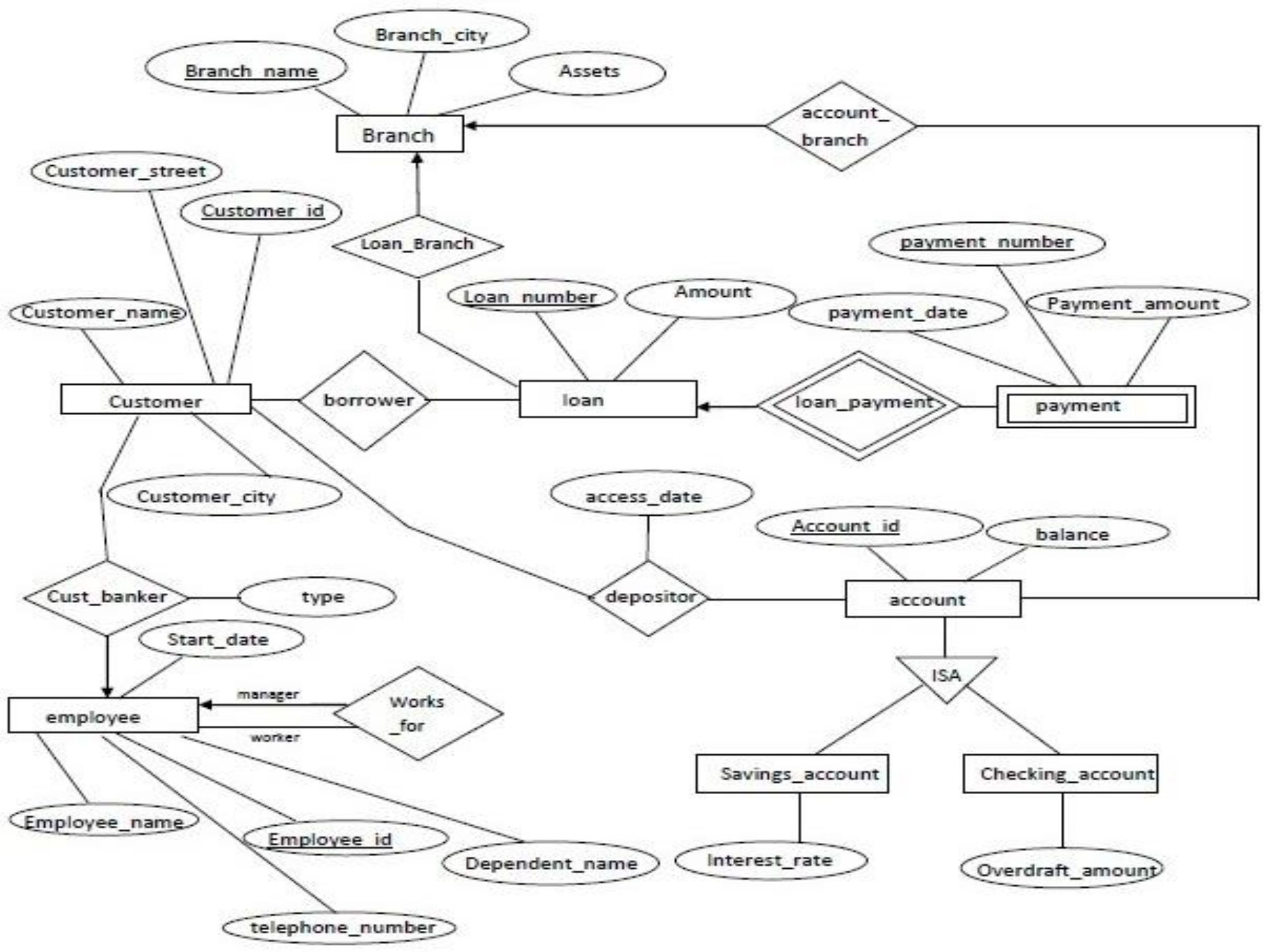

Department Name
Location
Department
1
1
1
Offers
Heade d by
Has
Course_ID
N
1
N
Course
N
Is taught by
1
Instructor
duration
M
course_name
Enrolled By
Instructor_ID
first_name
last_name
phone
N
Student
Student_ID
phone
first_name
last_name

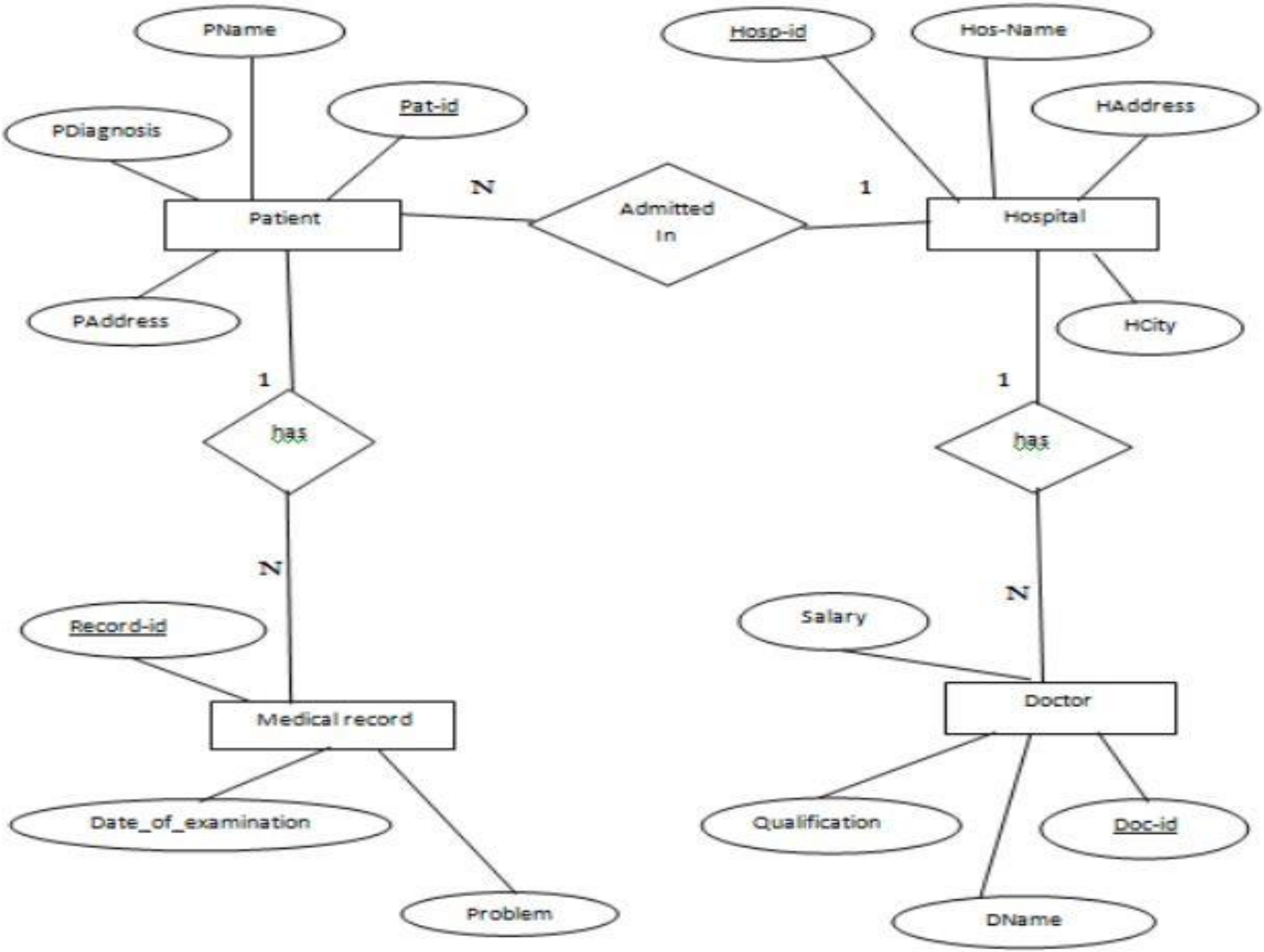
Voting System



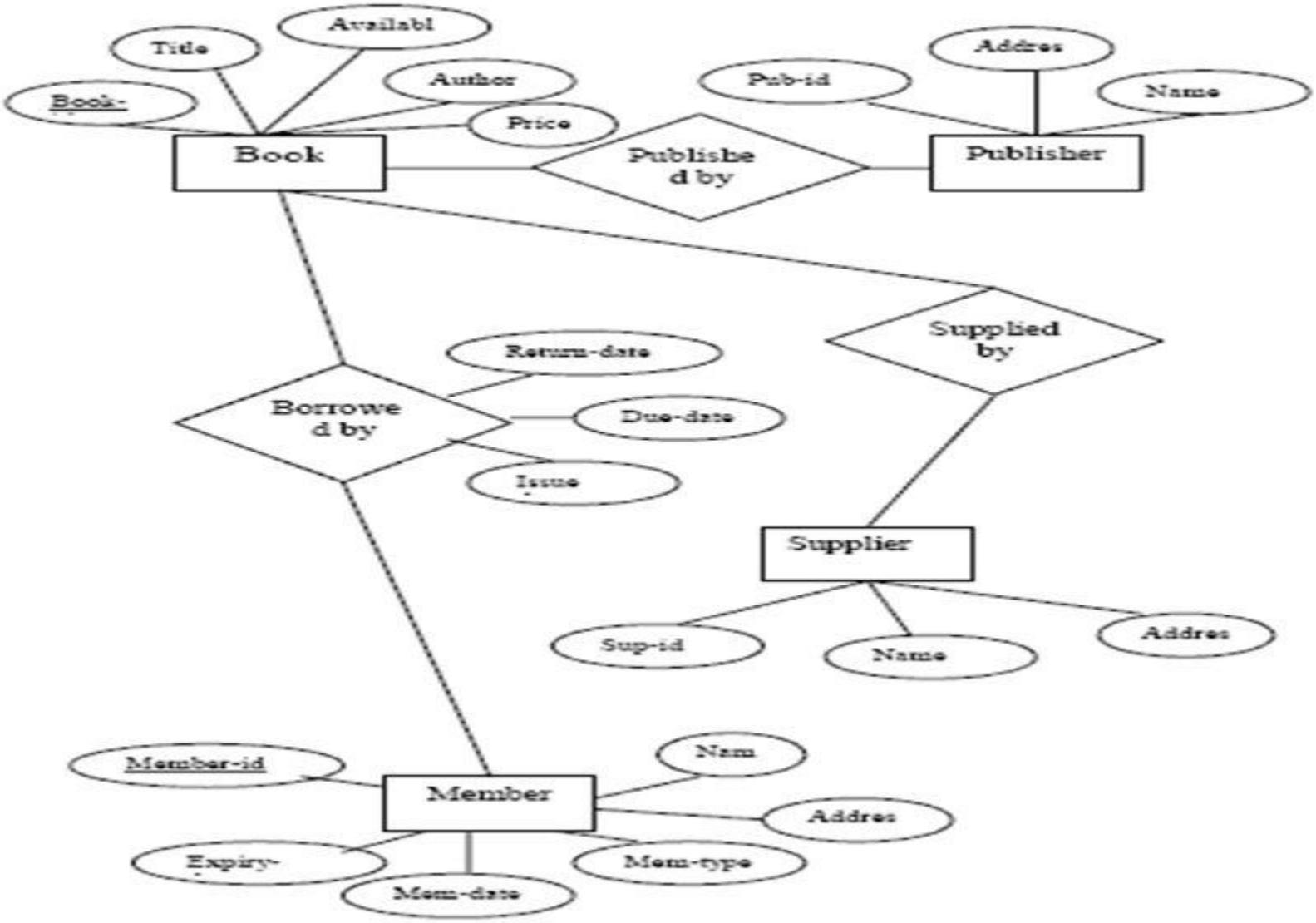
Bus Reservation System



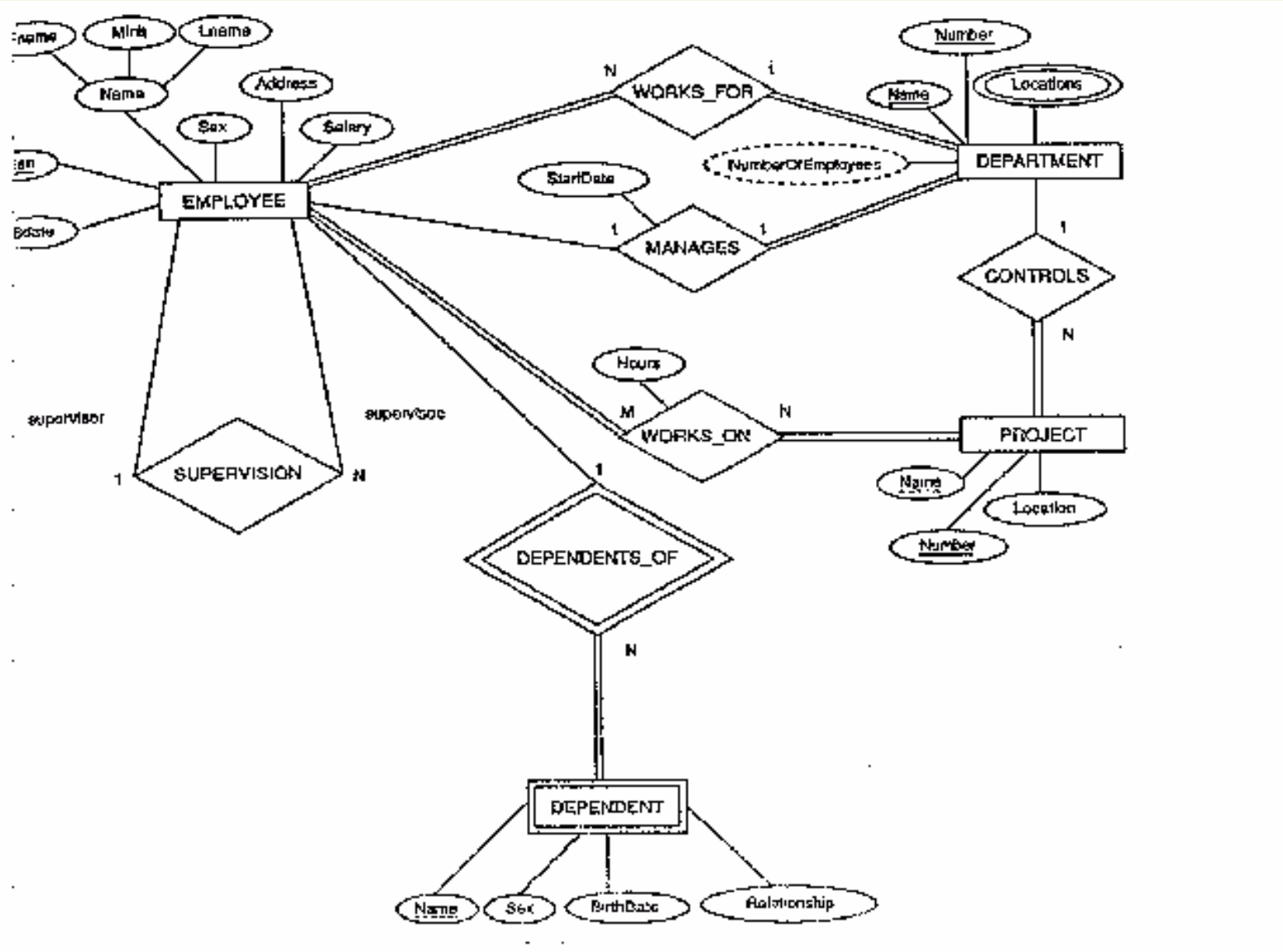

Branch_city
Branch_name
Assets
Branch
account_
branch
Customer_street
Customer_id
Loan_Branch
payment_number
Loan_number
Amount
payment_date
Payment_amount
Customer_name
Customer
borrower
loan
loan_payment
payment
Customer_city
access_date
Account_id
balance
Cust_banker
type
depositor
account
Start_date
ISA
manager
employee
Works
_for
worker
Savings_account
Checking_account
Employee_name
Employee_id
Dependent_name
Interest_rate
Overdraft_amount
telephone_number


PName
Pat-id
PDiagnosis
Patient
PAddress
N
Admitted In
1
Hosp-id
Hos-Name
HAddress
Hospital
HCity
1
has
N
Record-id
Medical record
Date_of_examination
Problem
1
has
N
Salary
Doctor
Qualification
Doc-id
DName

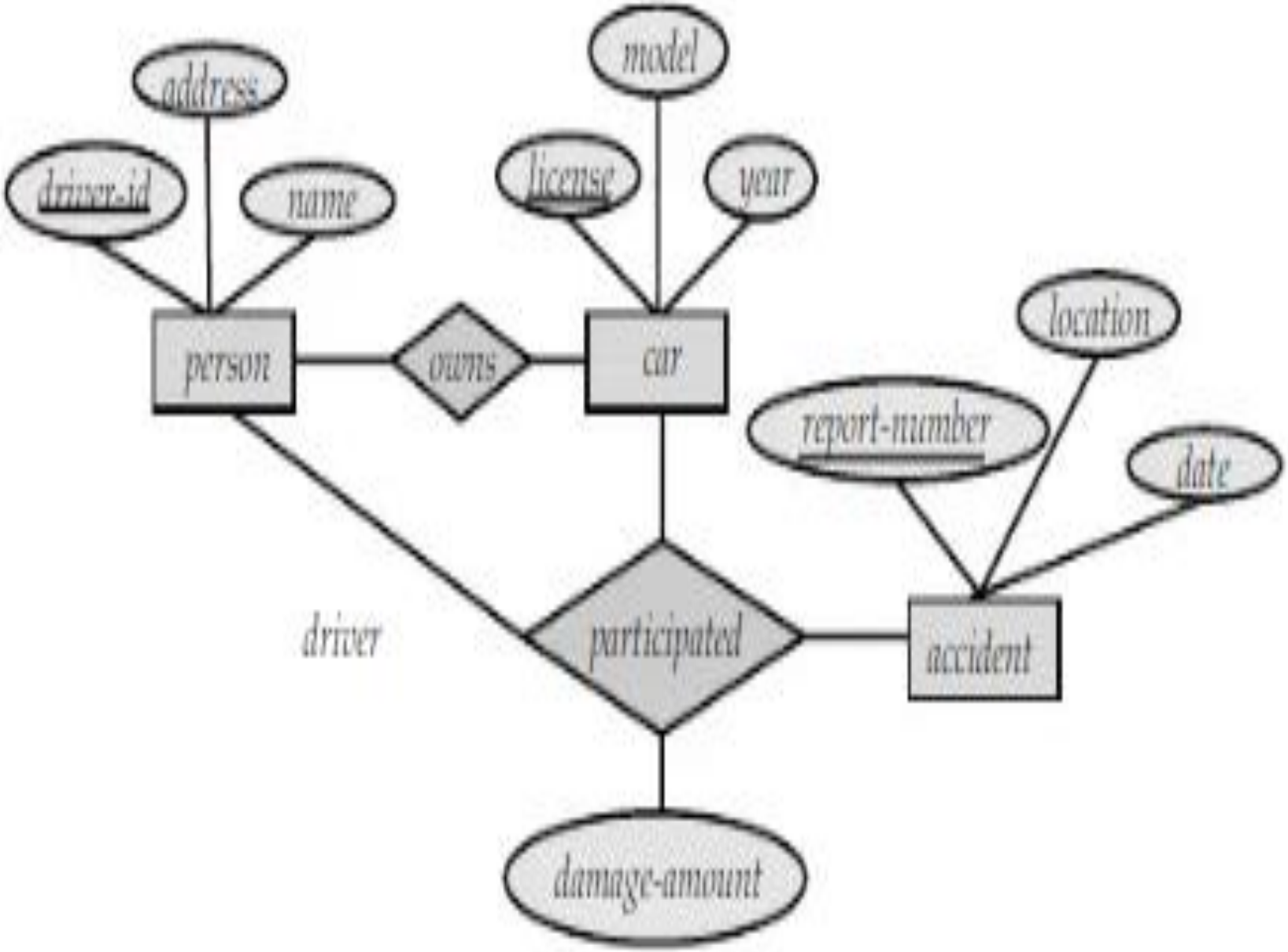
E-R Diagram for library management system (cont…)



1. COMPANY has departments
2. Department has name, number, manager
Manager is an employee
Manager has starting date
Department has several locations
3. Department controls projects
Project has name, number, location
4. Employee has name, social security, number, address, salary, sex, birthdate.
5. Employee is in one department but may work on several projects
6. Employees works for Hours on each project
7. Employee has supervisor
8. Employee has dependents with name, sex, birthdate, and relationship to employee.


Fname
Minit
Lname
Name
Address
Sex
Salary
Bdate
EMPLOYEE
N
WORKS_FOR
1
Number
Name
Locations
DEPARTMENT
NumberOfEmployees
StartDate
1
MANAGES
1
1
CONTROLS
N
Hours
M
WORKS_ON
N
PROJECT
Name
Location
Number
supervisor
supervisee
1
SUPERVISION
N
1
DEPENDENTS_OF
N
DEPENDENT
Name
Sex
BirthDate
Relationship

Construct an E-R diagram for a car insurance company whose customers own one or more cars each. Each car has associated with it zero to any number of recorded accidents


address
model
driver-id
name
license
year
person
owns
car
location
report-number
date
driver
participated
accident
damage-amount

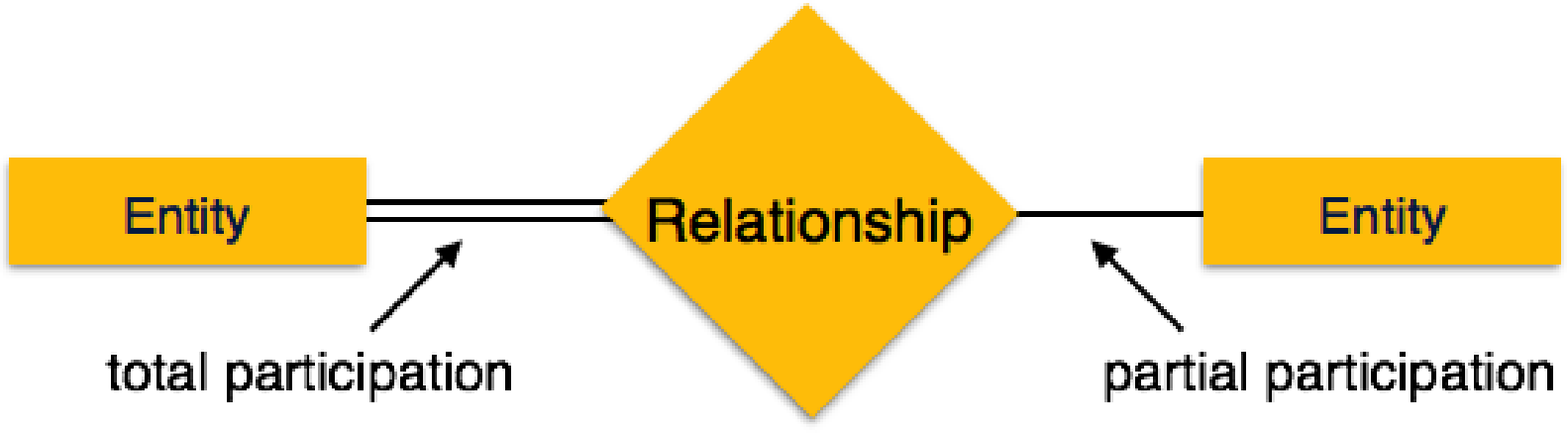
Consider a university database for the scheduling of classrooms for final exams. This database could be modeled as the single entity set *exam*, with attributes *course-name*, *section-number*, *room-number*, and *time*. Alternatively, one or more additional entity sets could be defined, along with relationship sets to replace some of the attributes of the *exam* entity set, as

- *course* with attributes *name*, *department*, and *c-number*
- *section* with attributes *s-number* and *enrollment*, and dependent as a weak entity set on *course*
- *room* with attributes *r-number*, *capacity*, and *building*

Participation Constraints

•**Total Participation** – Each entity is involved in the relationship. Every entity must participate. Total participation is represented by double lines. A bank account must belong to a customer.

•**Partial participation** – Not all entities are involved in the relationship. Some entities may not participate. Partial participation is represented by single lines. A student may or may not enroll in a course.

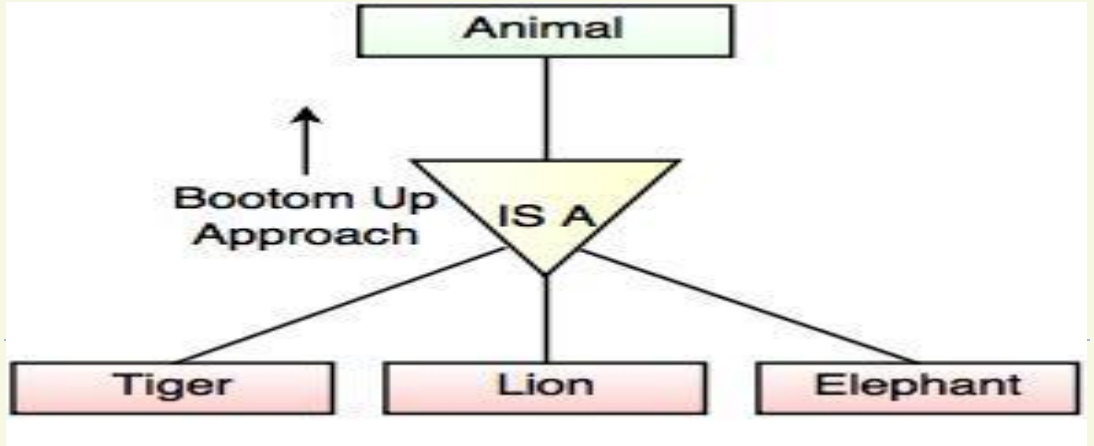


Extended E-R Diagram

Supertype and Subtype
•Supertype: A generic entity
•Subtype: Specialized entity derived from a supertype
Example: Employee → Teaching_Staff, Non_Teaching_Staff

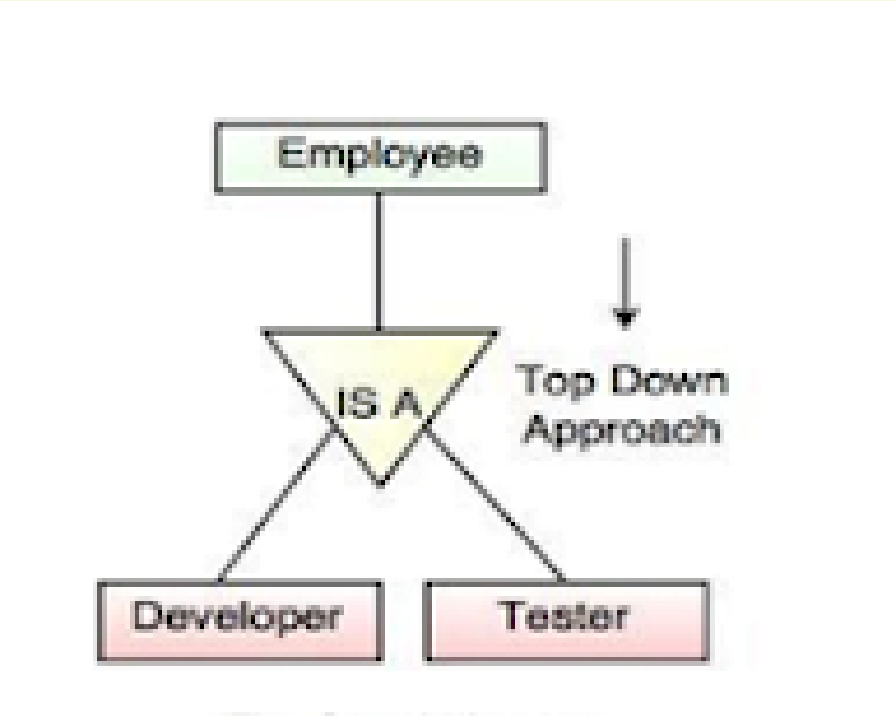
•**Generalization :** In generalization, a number of entities are brought together into one generalized entity based on their similar characteristics.

•Bottom-up process
•Combining similar entity types into a higher-level entity



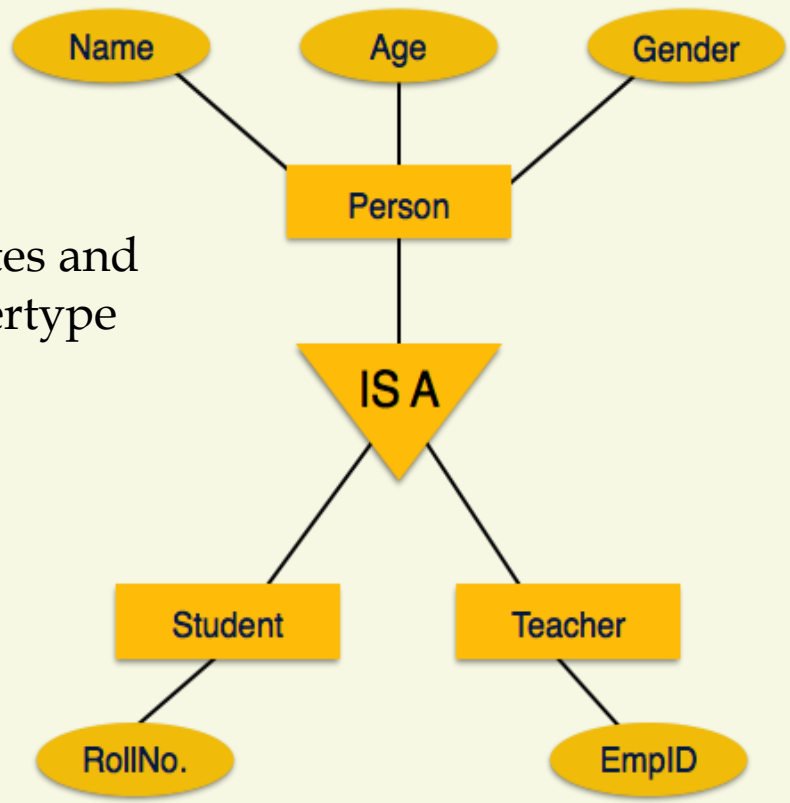
•**Specialization :** Specialization is the opposite of generalization. In specialization, a group of entities is divided into sub-groups based on their characteristics.

Top-down processDividing a supertype into subtypes based on distinguishing characteristics



Inheritance : It allows lower-level entities to inherit the attributes of higher-level entities.

Subtypes inherit attributes and relationships of the supertype



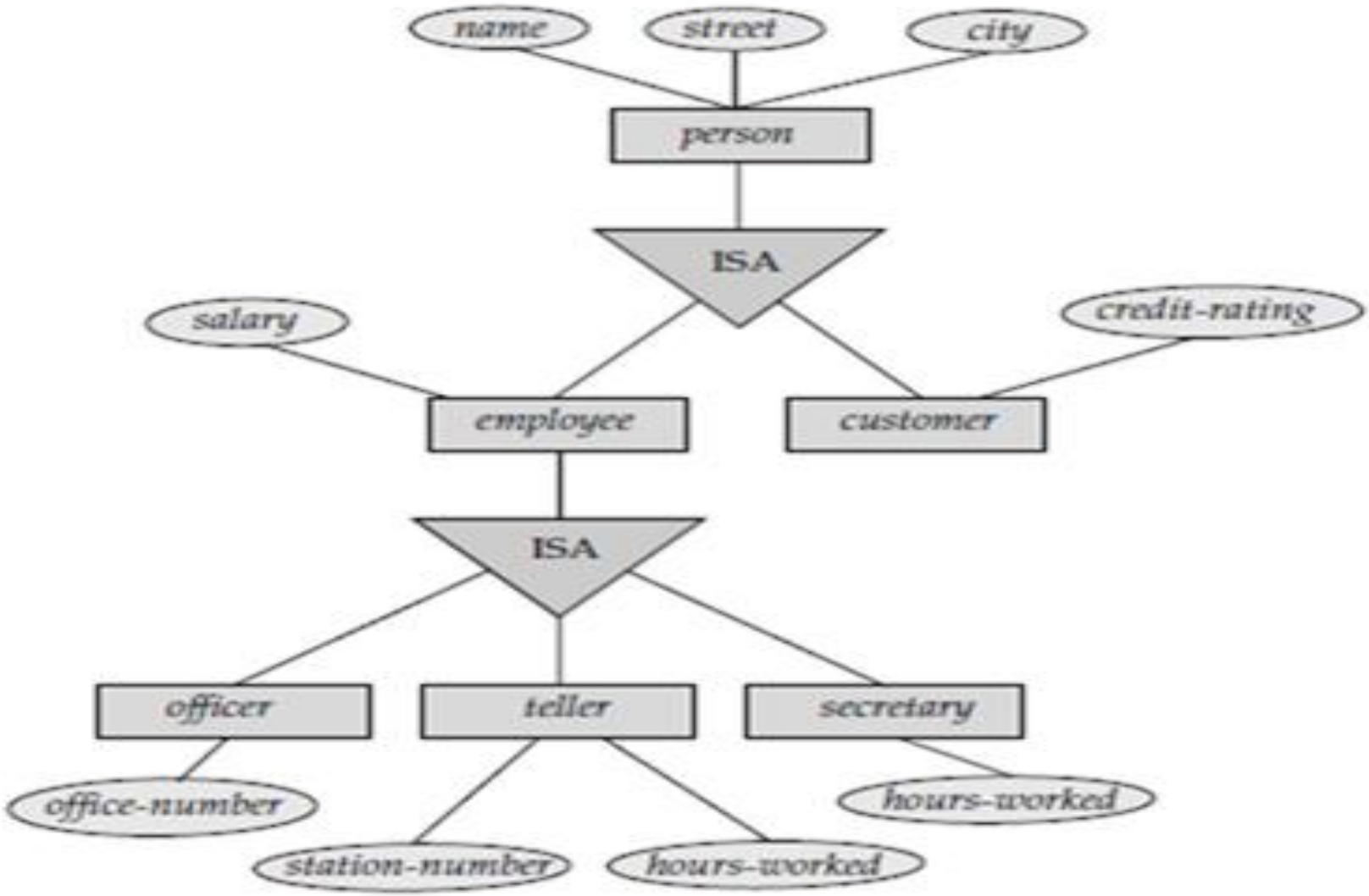
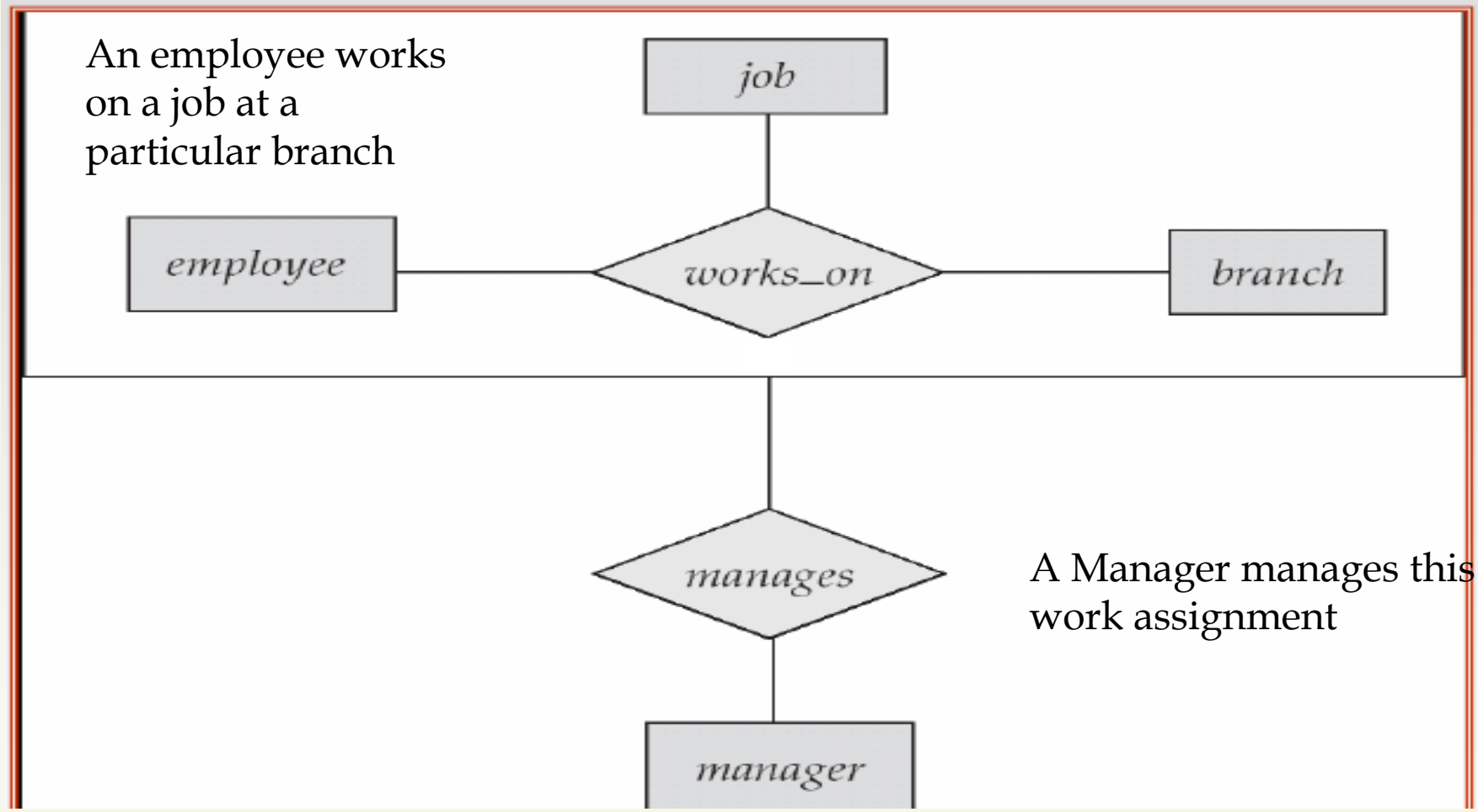


Figure 2.17 Specialization and generalization.

Aggregation

Aggregation treats a relationship as if it were an entity.

Represents a relationship between relationships
Treats a relationship set as an entity



Library management system

1.In the library Management system, the following entities and attributes can be identified.

2.*Book* -the set all the books in the library. Each book has a Book-id, Title, Author, Price, and Available (y or n) as its attributes.

3.Member-the set all the library members. The member is described by the attributes Member_id, Name, Street, Mem_type, Mem_date (date of membership), Expiry_date.

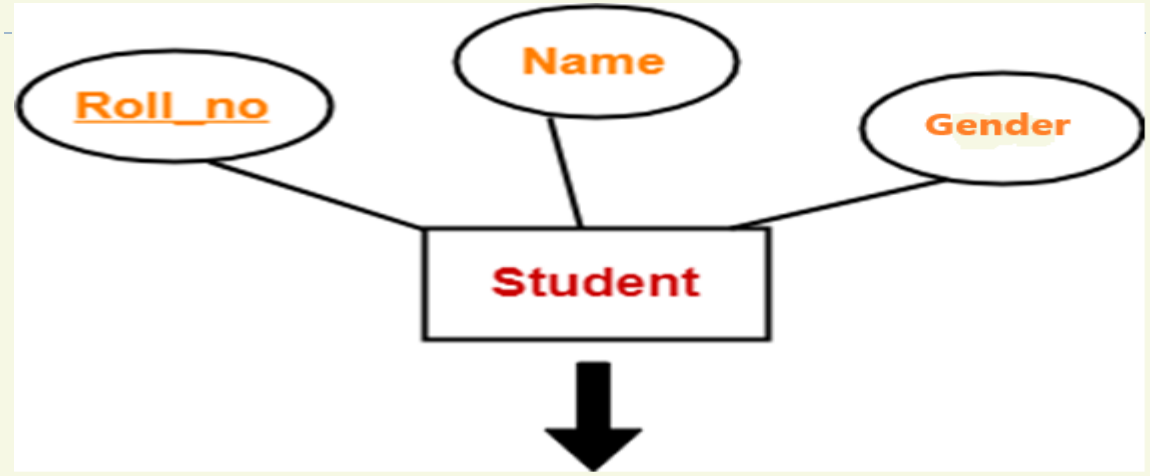
4.Publisher-the set of all the publishers of the books. Attributes of this entity are Pub_id, Name, Street, City, and Zip_code.

5.Supplier-the set of all the Suppliers of the books. Attributes of this entity are Sup_id, Name, Street, City, and Zip_code.

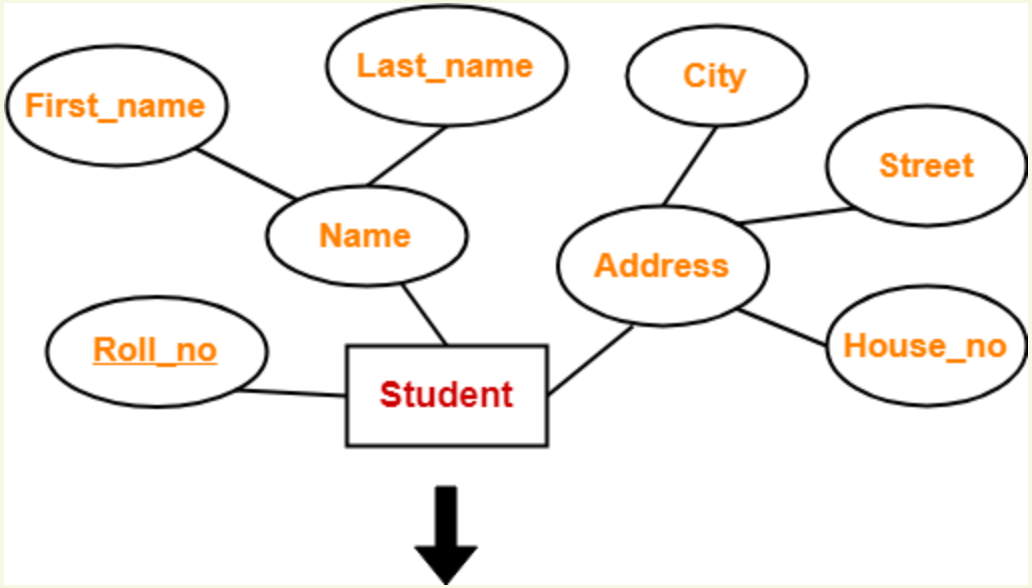
6.Assumptions: a publisher publishes a book. Supplier supplies book to library. Members borrow the book (only issue).

7.Return of book is not taken into account

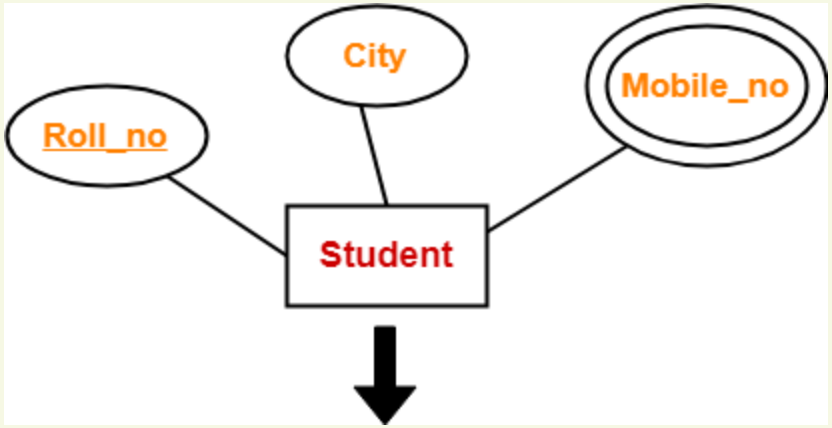
Converting ER diagram into Tables



Student (Roll_no , Name , Gender)

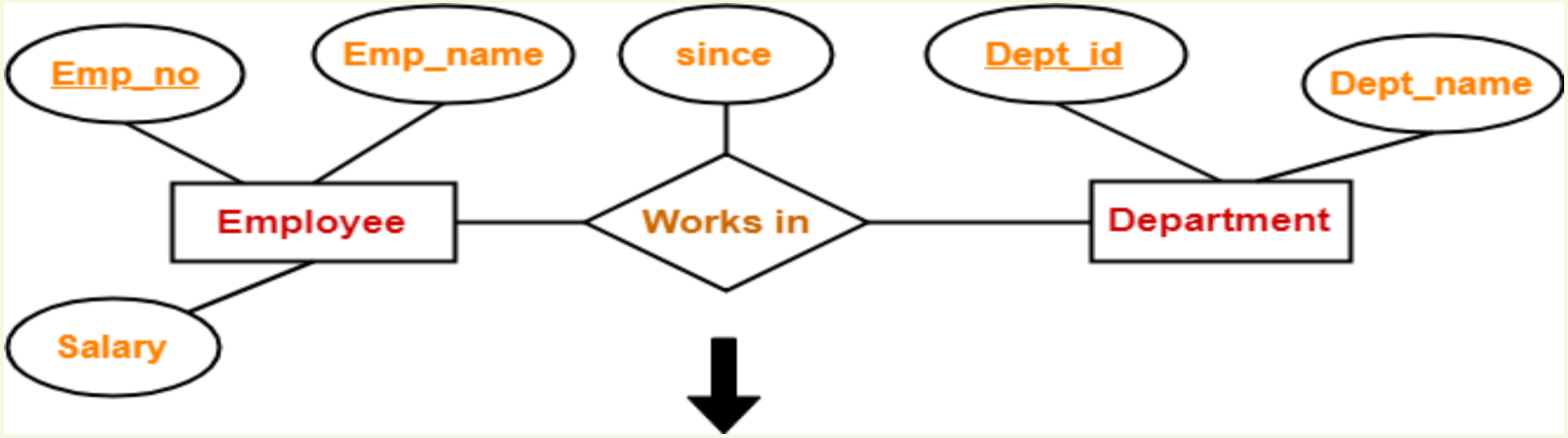


Student (Roll_no , First_name , Last_name , House_no , Street , City)



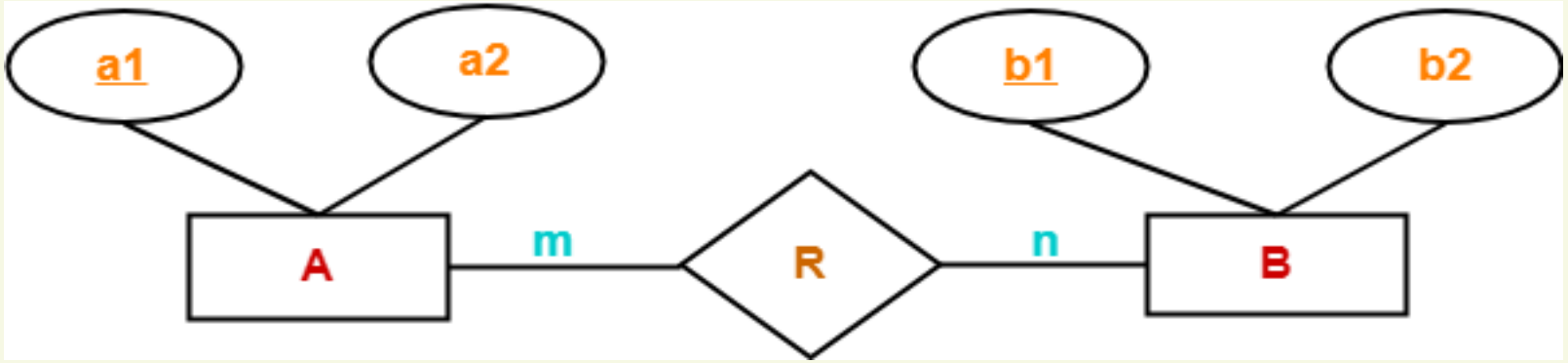
Roll_no	City

Roll_no	Mobile_no



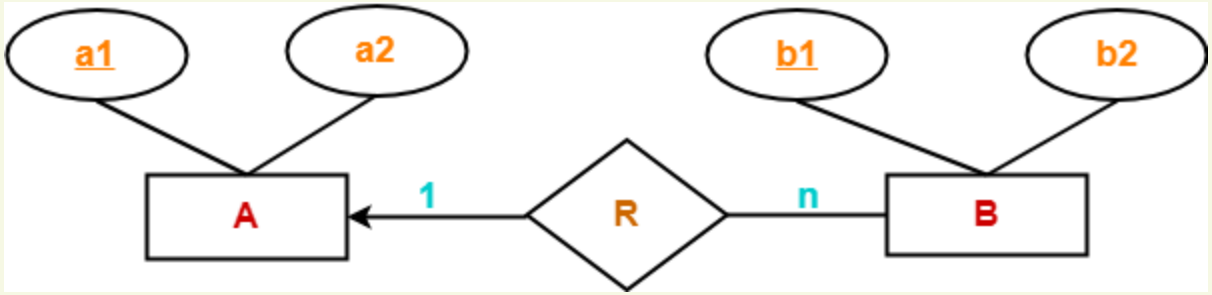
Emp_no	Dept_id	since

Binary Relationship With Cardinality Ratio m:n



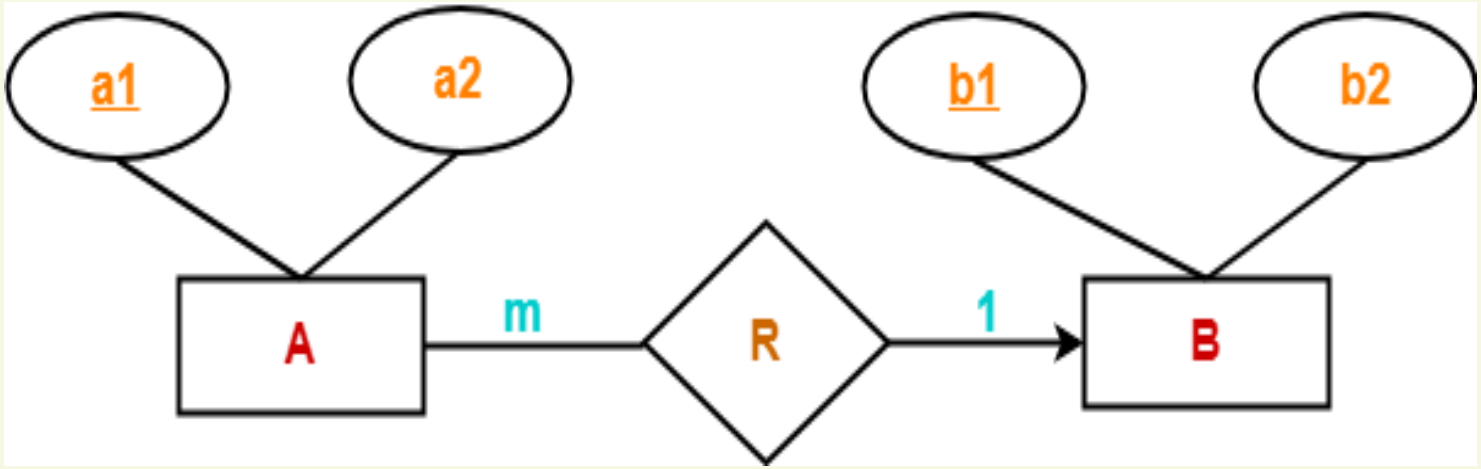
A (a1 , a2)
R (a1 , b1)
B (b1 , b2)

Binary Relationship With Cardinality Ratio 1:n



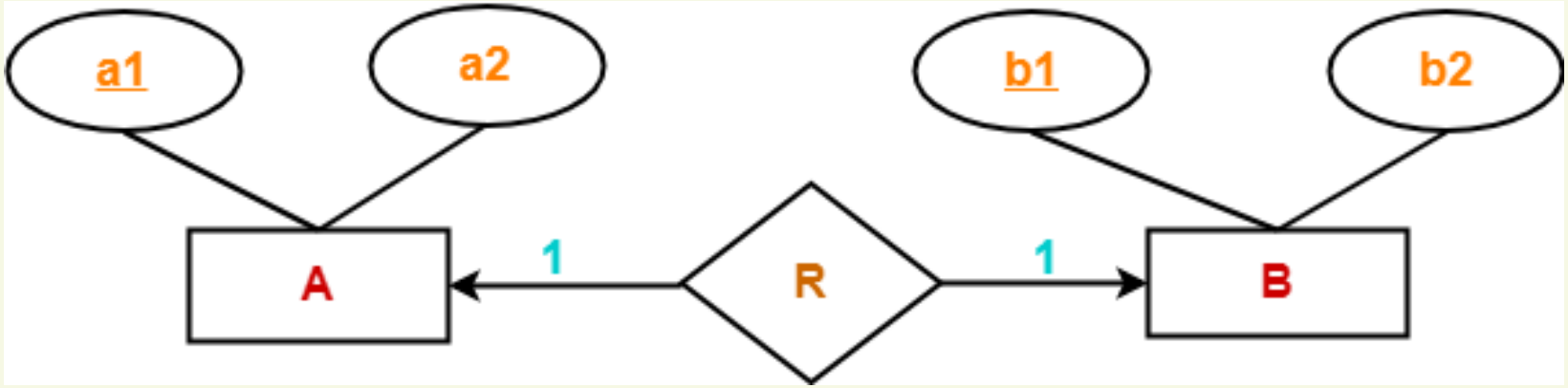
A (a1 , a2)
BR (a1 , b1 , b2)

Binary Relationship With Cardinality Ratio m:1



AR (a1 , a2 , b1)
B (b1 , b2)

Binary Relationship With Cardinality Ratio 1:1

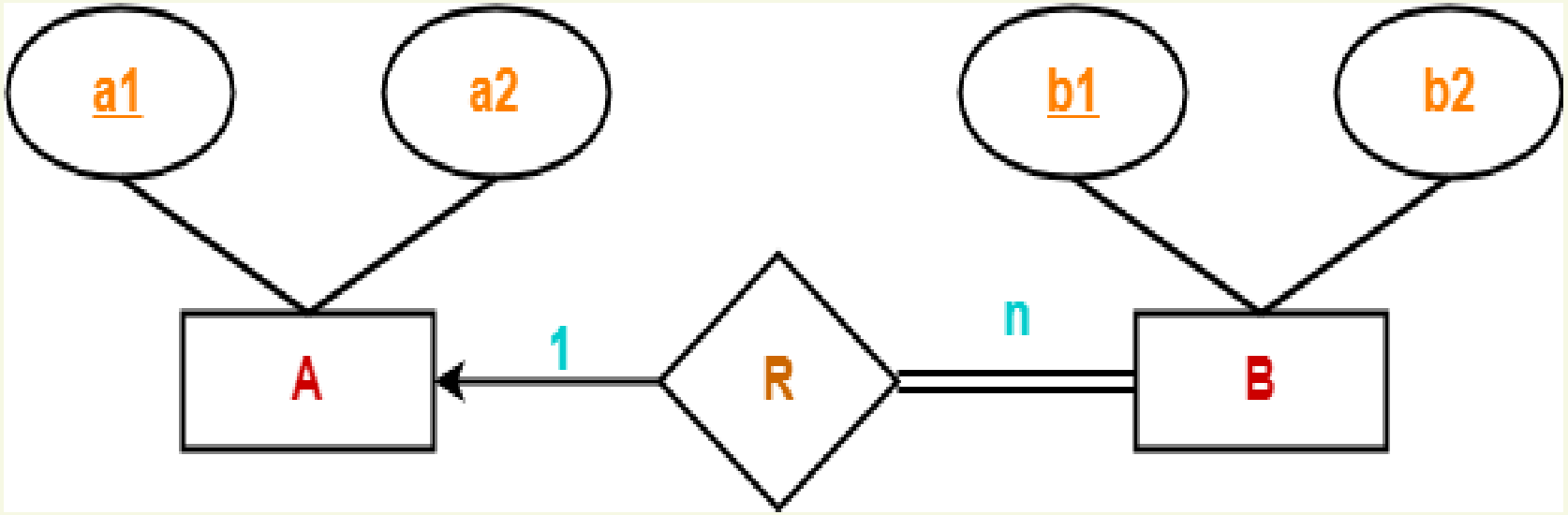


AR (a1 , a2 , b1)
B (b1 , b2)

OR

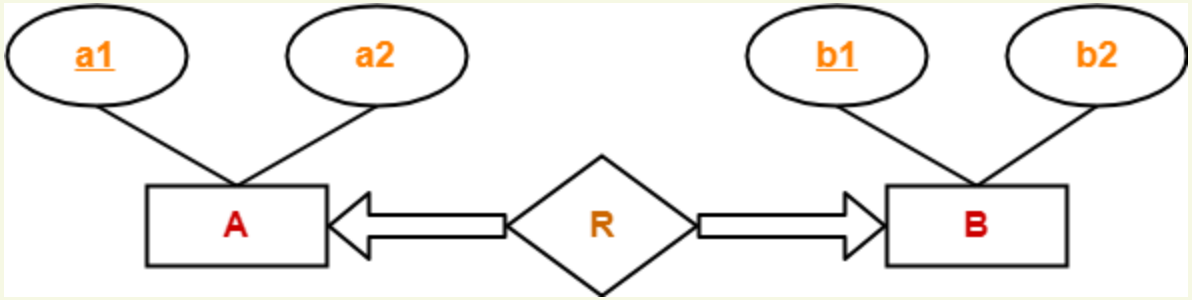
A (a1 , a2)
BR (a1 , b1 , b2)

Binary Relationship With Cardinality Constraint and Total Participation



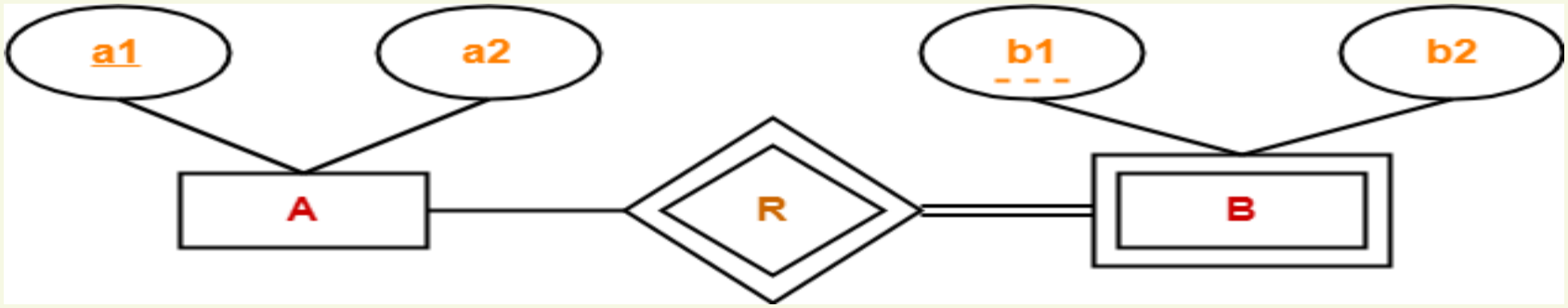
A (a1 , a2)
BR (a1 , b1 , b2)

Binary Relationship With Cardinality Constraint and Total Participation Constraint From Both Sides



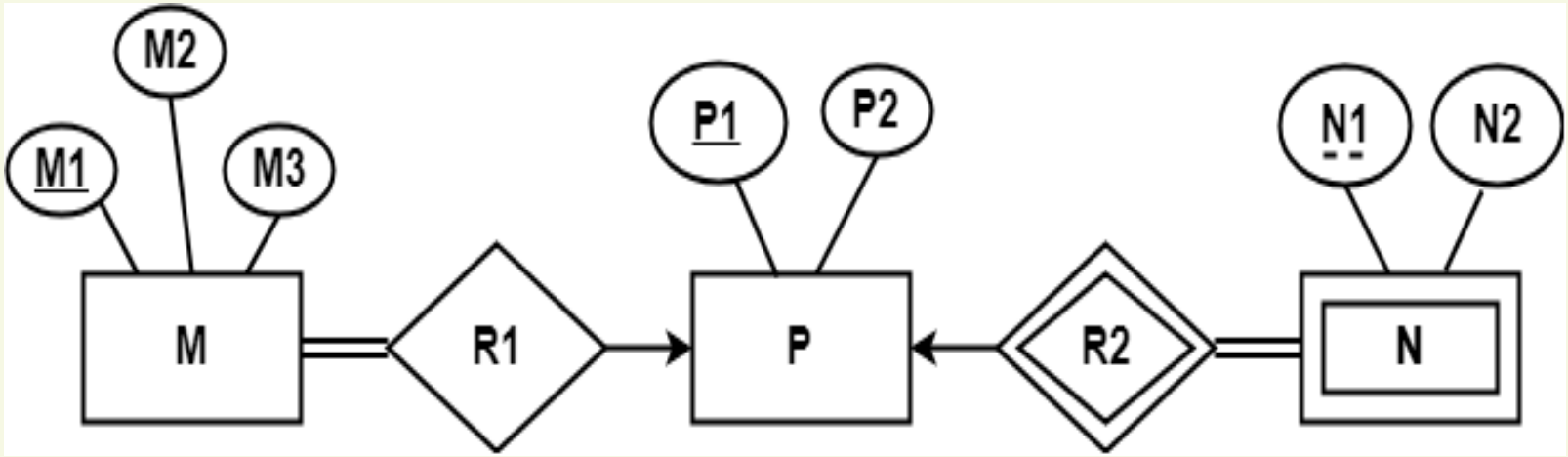
ARB (a1 , a2 , b1 , b2)

Binary Relationship With Weak Entity Set



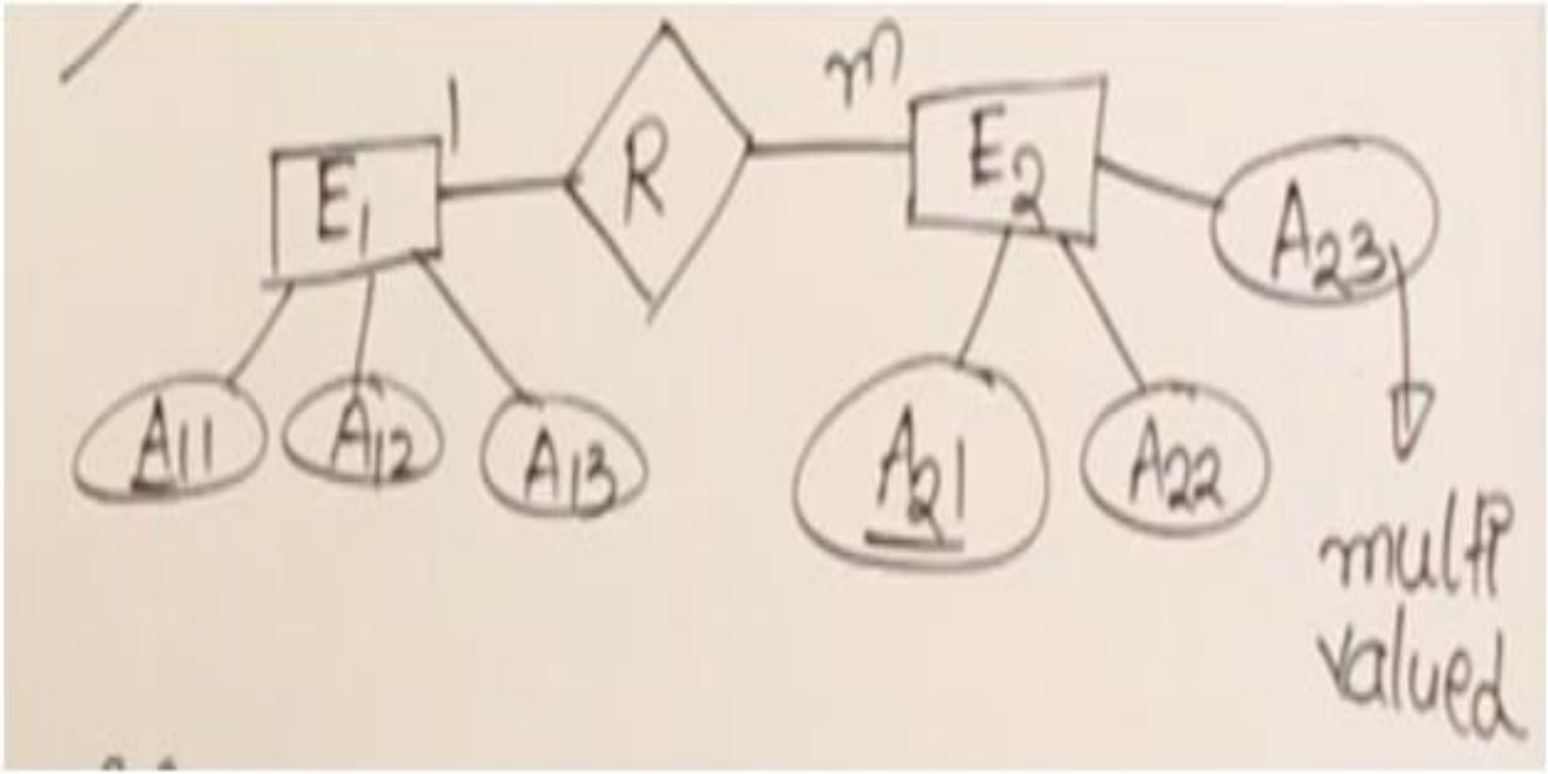
A (a1 , a2)
BR (a1 , b1 , b2)

PRACTICE PROBLEM:1

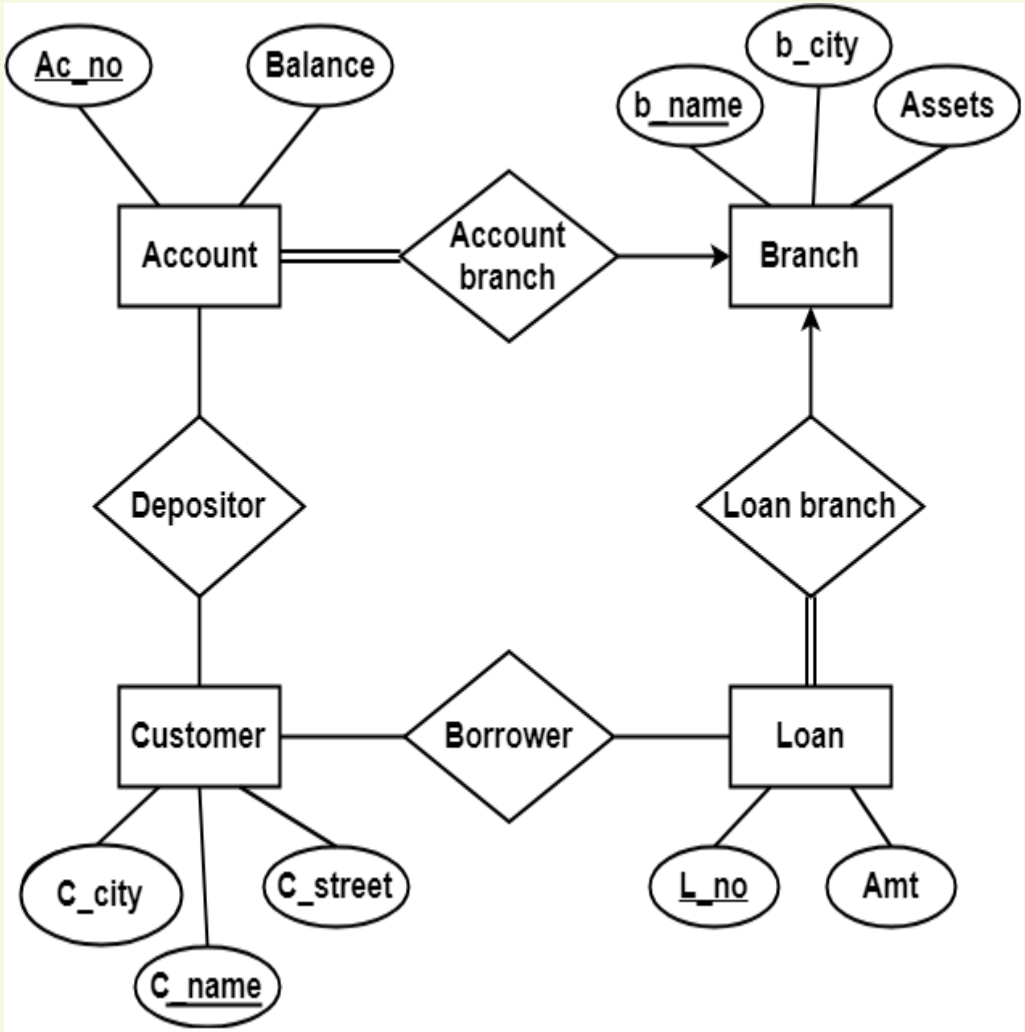


MR1 (M1 , M2 , M3 , P1)
P (P1 , P2)
NR2 (P1 , N1 , N2)

PRACTICE PROBLEM: 2



PRACTICE PROBLEM: 3



Account (Ac_no , Balance , b_name)
Branch (b_name , b_city , Assets)
Loan (L_no , Amt , b_name)
Borrower (C_name , L_no)
Customer (C_name , C_street , C_city)
Depositor (C_name , Ac_no)

GATE QUESTIONS

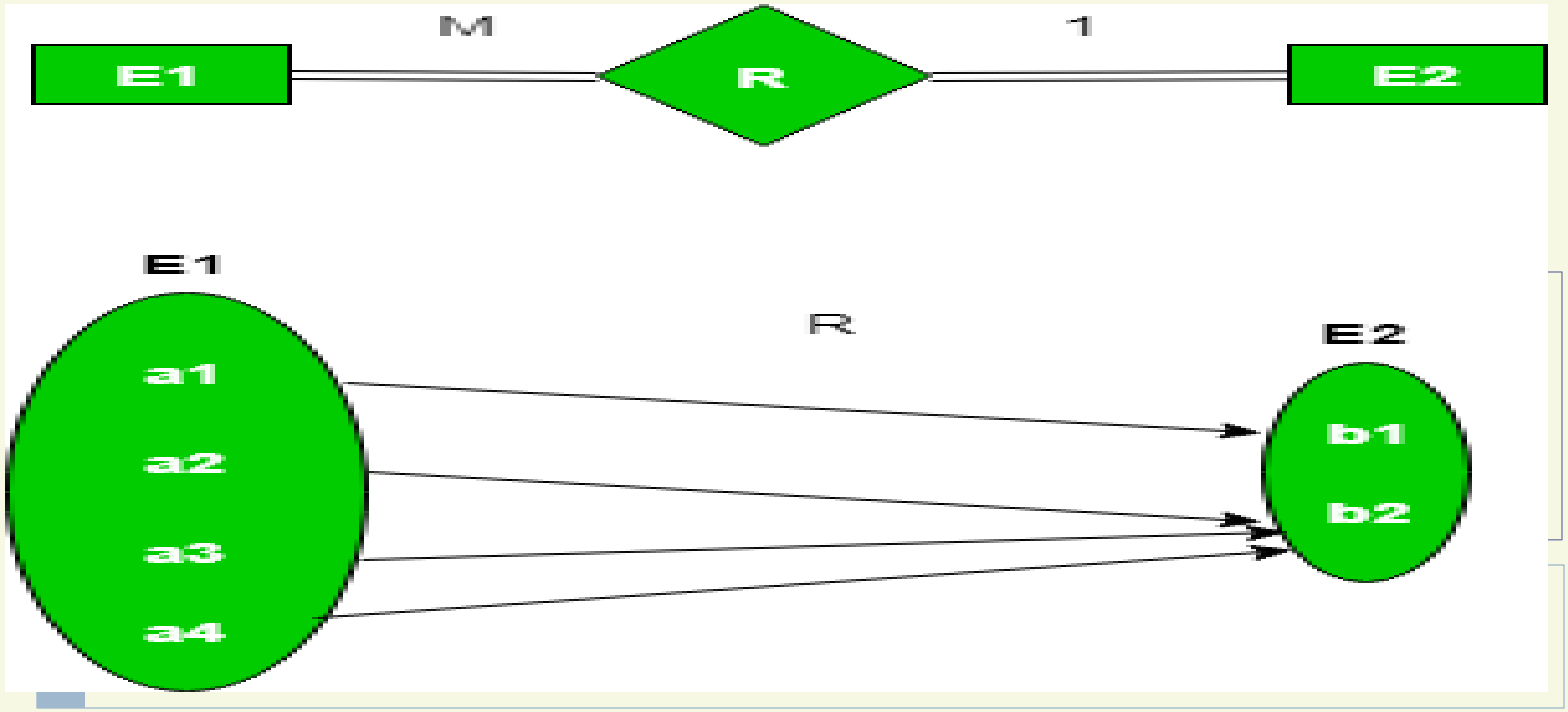
In an Entity-Relationship (ER) model, suppose R is a many-to-one relationship from entity set E1 to entity set E2. Assume that E1 and E2 participate totally in R and that the cardinality of E1 is greater that the cardinality of E2. Which one of the following is true about R?

A. Every entity in E1 is associated with exactly one entity in E2.

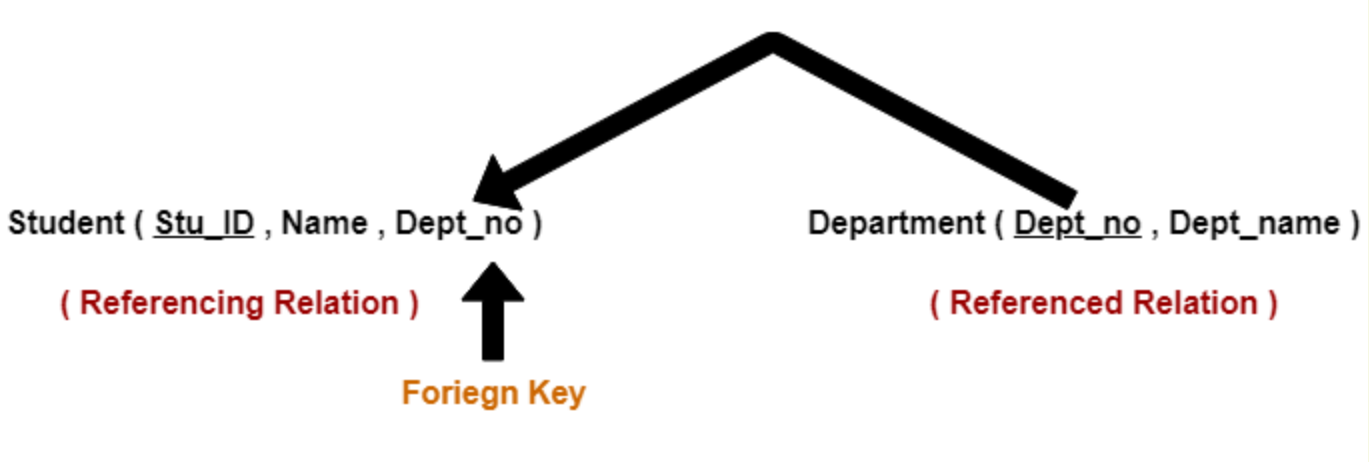
B. Some entity in E1 is associated with more than one entity in E2.

C. Every entity in E2 is associated with exactly one entity in E1.

D. Every entity in E2 is associated with at most one entity in E1


M
1
E1
R
E2
E1
R
E2
a1
a2
a3
a4
b1
b2

Foreign Key


Student (Stu_ID , Name , Dept_no)
(Referencing Relation)
Foriegn Key
Department (Dept_no , Dept_name)
(Referenced Relation)

1. Super Key A super key is any set of one or more attributes that can uniquely identify a record.

2. Candidate KeyA candidate key is a minimal super key (no extra attributes).

3. Primary Key A primary key is the candidate key chosen to identify records uniquely.

4. Alternate Key Candidate keys not chosen as the primary key.

5. Foreign KeyA foreign key links one table to another.

A minimal super key is called a:

A. Primary Key
B. Candidate Key
C. Foreign Key
D. Composite Key

Which of the following can contain NULL values?

A. Primary Key

B. Candidate Key

C. Unique Key

D. Super Key

A key formed by combining two or more attributes is known as:

A. Super Key

B. Foreign Key

C. Composite Key

D. Alternate Key